AWS CLOUD DEVOPS TRAINING | PLATFORM ENGINEERING

Lab 1: Terraform

Spinning Up an Apache Web Server on AWS EC2 Using Terraform

Prepared by: Samuel Akosa Onyejekwe

GitHub: github.com/samuelakosaonyejekwe

Repository: github.com/samuelakosaonyejekwe/cba_terraform

AWS Region: eu-west-2 (Europe – London)

Date completed: 1 November 2026

Tools: Terraform v1.14.6, HashiCorp AWS provider v6.71.0, AWS CLI v2, Git, GitHub CLI, VS Code + WSL Ubuntu

Contents

Contents

1. Overview

Objective

Learning outcomes I achieved

What I built

Request flow

2. My Environment and Prerequisites

3. Task 1 – Create a GitHub Repository

4. Task 2 – Open VS Code (WSL) and Clone the Repository

5. Task 3 – Create the Terraform Module

variables.tf

main.tf – provider, security group and instance

terraform init

6. Task 4 – Deploy the Terraform Module to AWS

terraform plan

terraform apply

Verifying in the AWS Console

Inspecting the Terraform state

7. Task 5 – Deploy the Apache Web Server

A problem I solved: user data only runs on first boot

Testing the web server

8. Task 6 – Push the Terraform Files to GitHub

9. Task 7 – Destroy the Resources

10. Challenges, Decisions and Key Learnings

Challenges and how I resolved them

What I learned

Possible improvements

11. Evidence Index

(Right-click the table and choose “Update Field” in Word to refresh page numbers.)

1. Overview

In this lab I wrote my first Terraform module and used it to provision real infrastructure on AWS. The module creates a security group and an EC2 instance. A bootstrap (user data) script then turns the instance into an Apache web server that serves a “Hello World” page over the public internet. I version-controlled the code in a public GitHub repository and destroyed every resource at the end so that I would not incur any cost.

Objective

- Create and run a Terraform module on AWS.

Learning outcomes I achieved

- Created a Terraform module (provider, variables, security group and EC2 instance).
- Spun up an Apache server on EC2 using Terraform and user data.
- Deployed the module to AWS, verified it in the AWS Console, CLI and a browser, then tore it down with terraform destroy.

What I built

Component	Terraform address	Configuration
Provider	provider "aws"	region = var.region (eu-west-2)
Security group	aws_security_group.cba_tf_sg	Name cba_tf_sg; inbound TCP 22 (SSH) and TCP 80 (HTTP) from 0.0.0.0/0; all outbound
EC2 instance	aws_instance.cba_tf_instance	Name CBATerraformInstance; Amazon Linux 2 AMI ami-01dfd9fb3a7b119e0; t3.micro; key pair cba-akosa-keypair; default VPC
User data	install_apache.sh	yum installs httpd, starts it and writes “Hello World from <hostname>” to /var/www/html/index.html

Request flow

Browser → Internet → public IP of EC2 instance → security group cba_tf_sg (allows TCP 80) → Apache httpd on Amazon Linux 2 → index.html (“Hello World from ip-172-31-15-40…”).

2. My Environment and Prerequisites

Item	Value I used
Workstation	Windows + WSL 2 (Ubuntu), VS Code connected to WSL
Terraform	v1.14.6 (linux_amd64)
AWS provider	hashicorp/aws v6.71.0 (installed by terraform init)
AWS identity	IAM user cba_admin (credentials configured in the AWS CLI)
Region	eu-west-2 – Europe (London)
VPC	Default VPC vpc-0dd7c25d7f1fb2785
Key pair	cba-akosa-keypair (existing key pair in my account)
AMI	ami-01dfd9fb3a7b119e0 – amzn2-ami-hvm-2.0.20261102.0-x86_64-gp2 (Amazon Linux 2, as the lab requires)
GitHub	samuelakosaonyejekwe, authenticated with the GitHub CLI

Before starting I confirmed that my GitHub CLI was authenticated (Figure 1). I also looked up the latest Amazon Linux 2 AMI for London from the public SSM parameter /aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2, and listed my existing key pairs so I could plug real values into variables.tf.

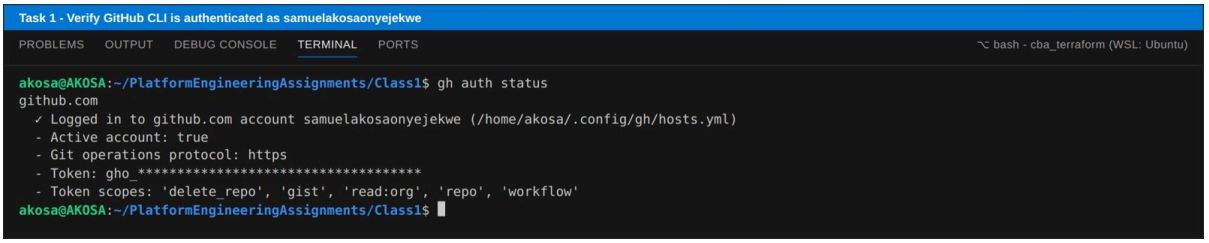


Figure 1: *GitHub CLI authenticated as samuelakosaonyejekwe*

3. Task 1 – Create a GitHub Repository

I created a new public repository named cba_terraform under my GitHub account, with a README file and the Terraform .gitignore template, exactly as the lab specifies. I did this from the terminal with the GitHub CLI, which is the command-line equivalent of clicking + → New repository, ticking Add a README, and choosing the Terraform .gitignore.

1. gh repo create cba_terraform --public --add-readme --gitignore Terraform
2. I copied the HTTPS link that GitHub returned:
   https://github.com/samuelakosaonyejekwe/cba_terraform

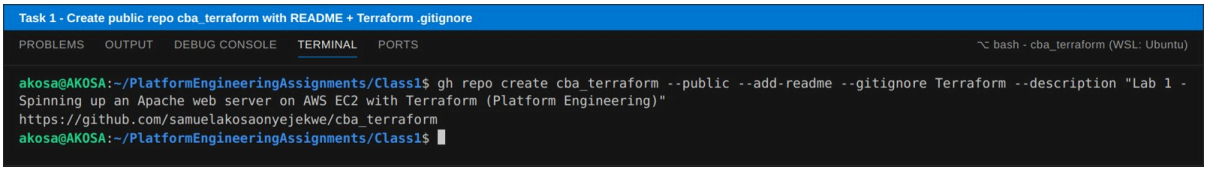


Figure 2: *Creating the public cba_terraform repository with a README and Terraform .gitignore*

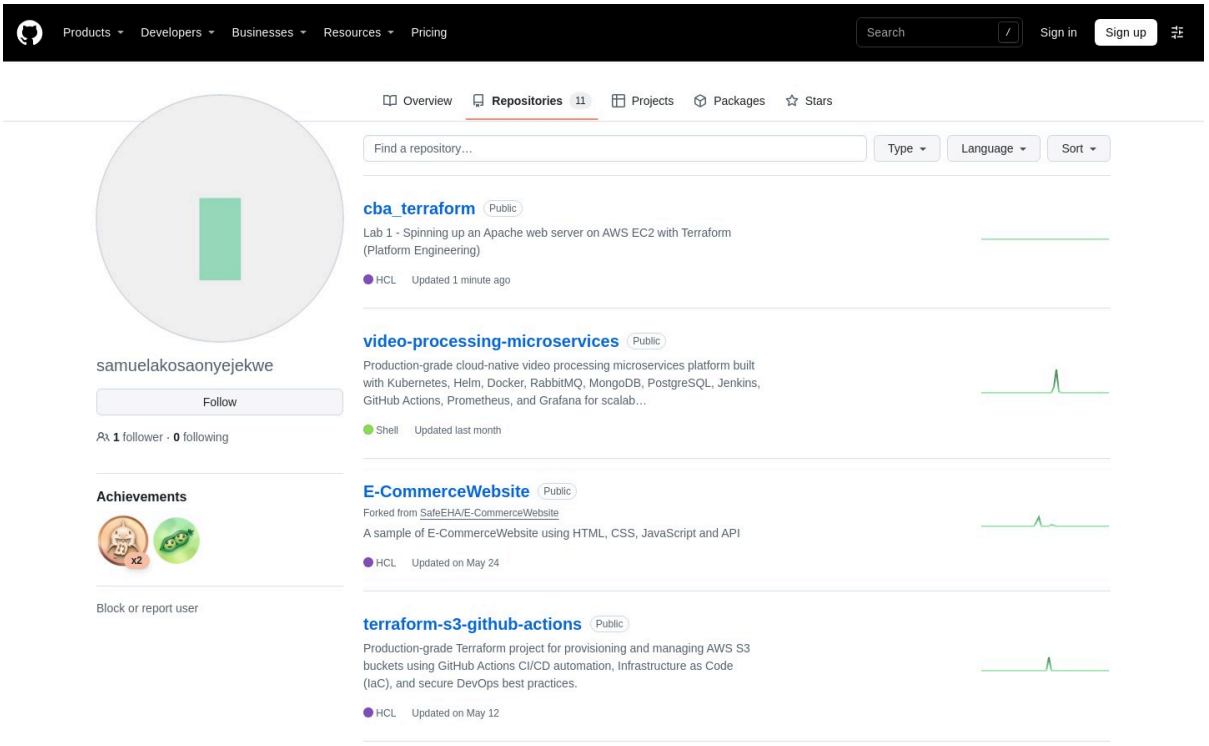


Figure 3: *cba_terraform listed (Public) on my GitHub profile*

The Terraform .gitignore matters. It stops terraform.tfstate, the .terraform/ provider cache and *.tfvars files from being committed. Those files can contain resource IDs and secrets.

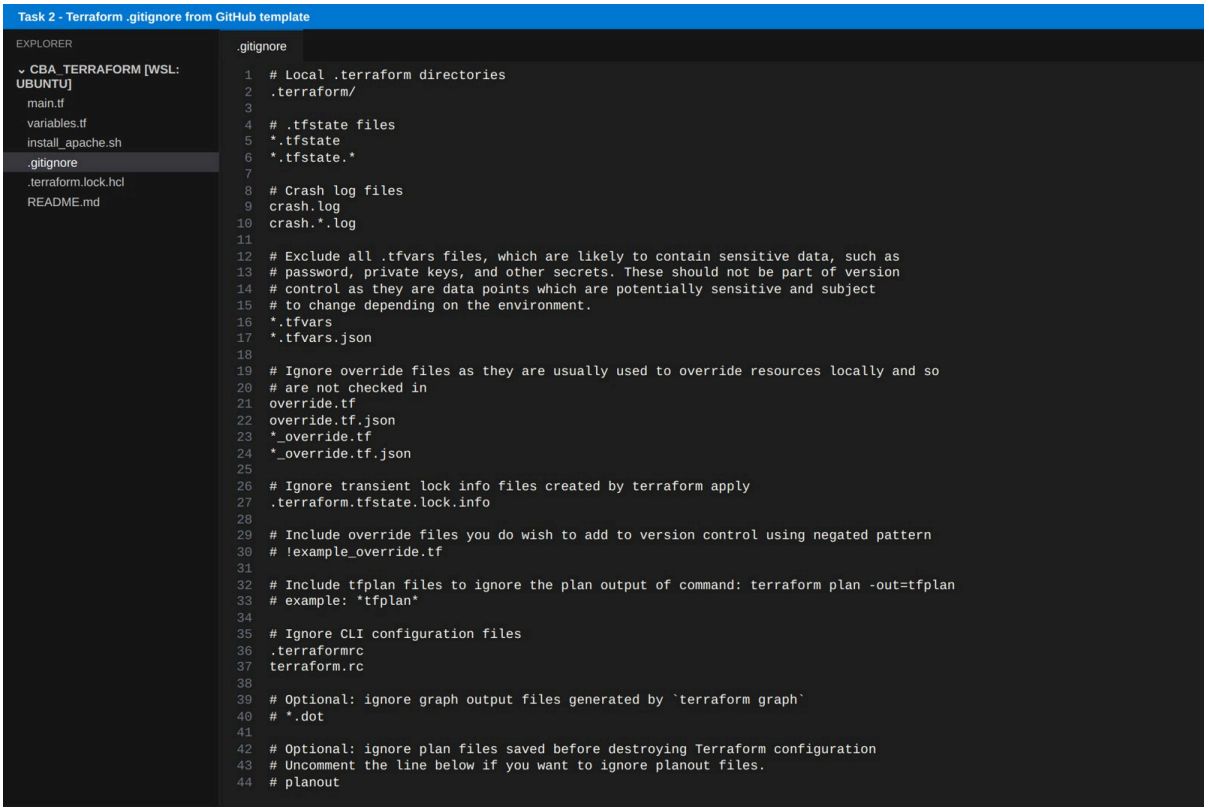


Figure 4: *The Terraform .gitignore template added by GitHub*

4. Task 2 – Open VS Code (WSL) and Clone the Repository

Terraform is installed in my Ubuntu (WSL) environment, so I worked in VS Code connected to WSL: Remote Window → Connect to WSL → Terminal. In the terminal I cloned the repository and opened the cba_terraform folder in the Explorer. I also installed the HashiCorp Terraform extension for syntax highlighting and IntelliSense.

3. git clone https://github.com/samuelakosaonyejekwe/cba_terraform.git

4. I changed into the folder and checked its contents (ls -la): .git, .gitignore and README.md.

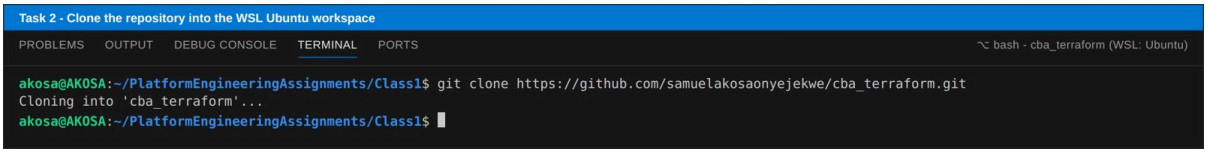


Figure 5: *Cloning my repository into the WSL workspace*

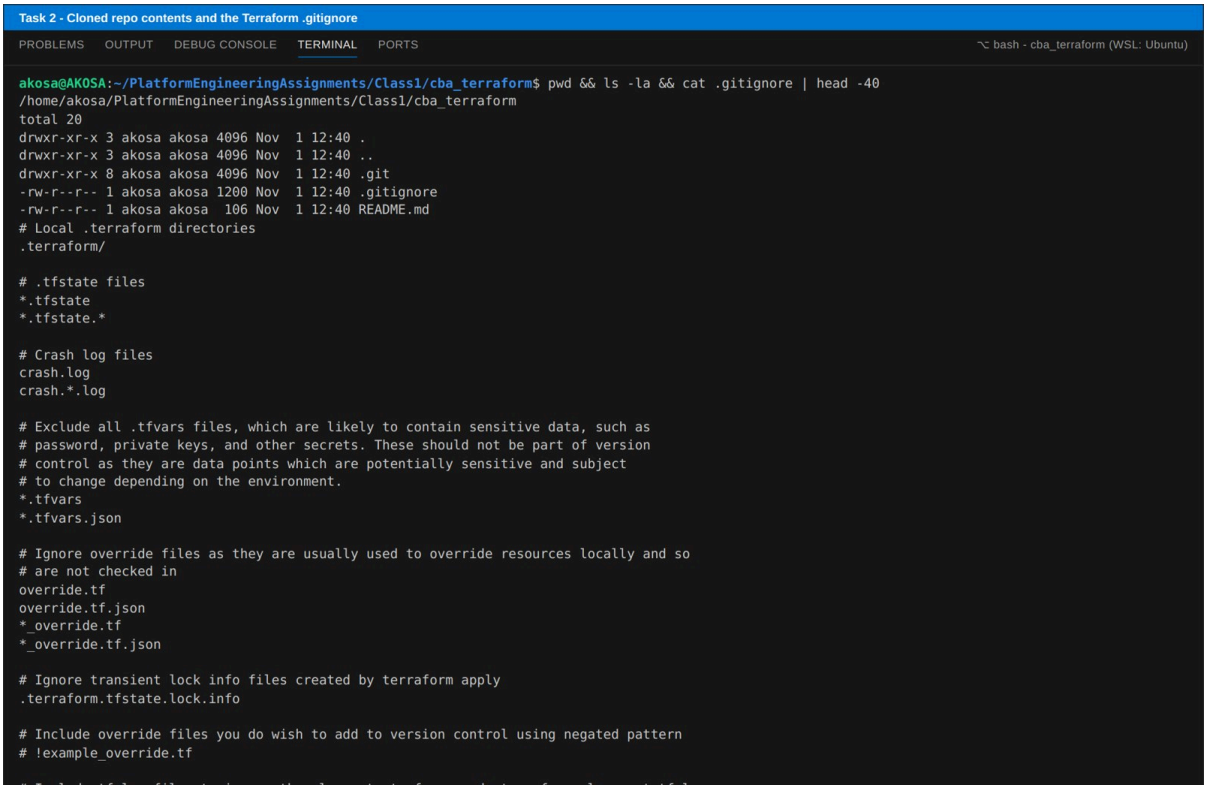


Figure 6: *Cloned repo contents and the start of the Terraform .gitignore*

5. Task 3 – Create the Terraform Module

Inside cba_terraform I created two files, variables.tf and main.tf, using the course reference scripts (terraform_labs_scripts/lab_01) as the starting point and replacing the placeholders with my own AWS values.

variables.tf

- **region** – eu-west-2 (London), the region where my resources are created.
- **instance_type** – t3.micro. The lab uses t2.micro, but when I queried my account t2.micro was no longer listed as Free Tier eligible, while t3.micro was. I chose t3.micro so the lab stays free.
- **instance_ami** – ami-01dfd9fb3a7b119e0, an Amazon Linux 2 AMI (the lab says not to use the default Amazon Linux 2023 AMI).
- **vpc_id** – empty, so AWS uses my default VPC.
- **key_name** – cba-akosa-keypair, a key pair that already exists in my account, for SSH access.

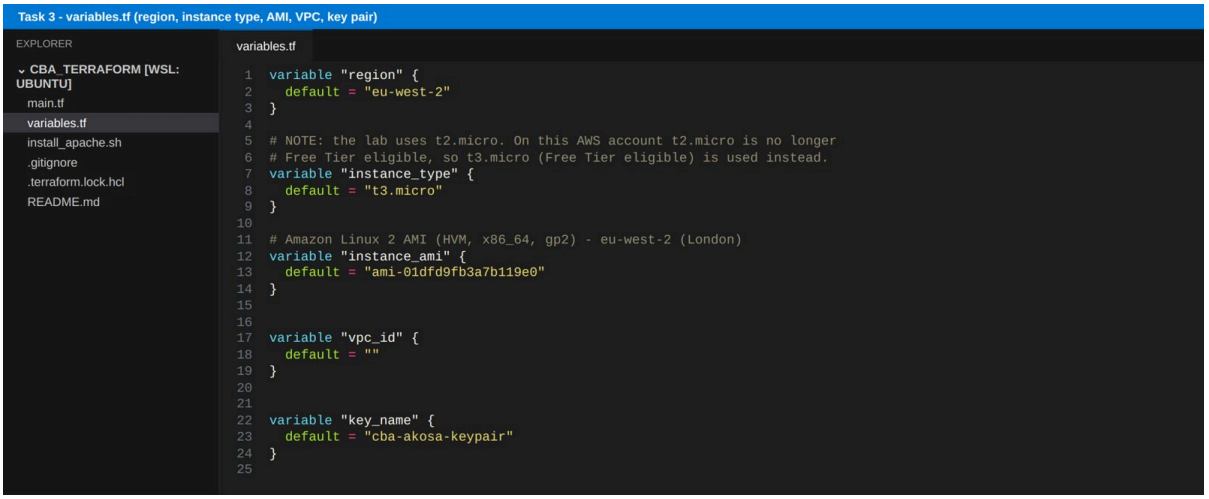


Figure 7: *My variables.tf with real values from my AWS account*

main.tf – provider, security group and instance

main.tf configures the AWS provider with region = var.region. It then defines the security group cba_tf_sg, which acts as a virtual firewall: two ingress rules (TCP 22 for SSH and TCP 80 for HTTP from 0.0.0.0/0) and one egress rule that allows all outbound traffic (protocol -1). Finally it defines the EC2 instance cba_tf_instance. The instance takes its type, AMI and key pair from variables, and attaches the security group by referencing aws_security_group.cba_tf_sg.name. That reference also tells Terraform to create the security group first. I added an Owner tag with my GitHub username so the resources are easy to identify in the console.

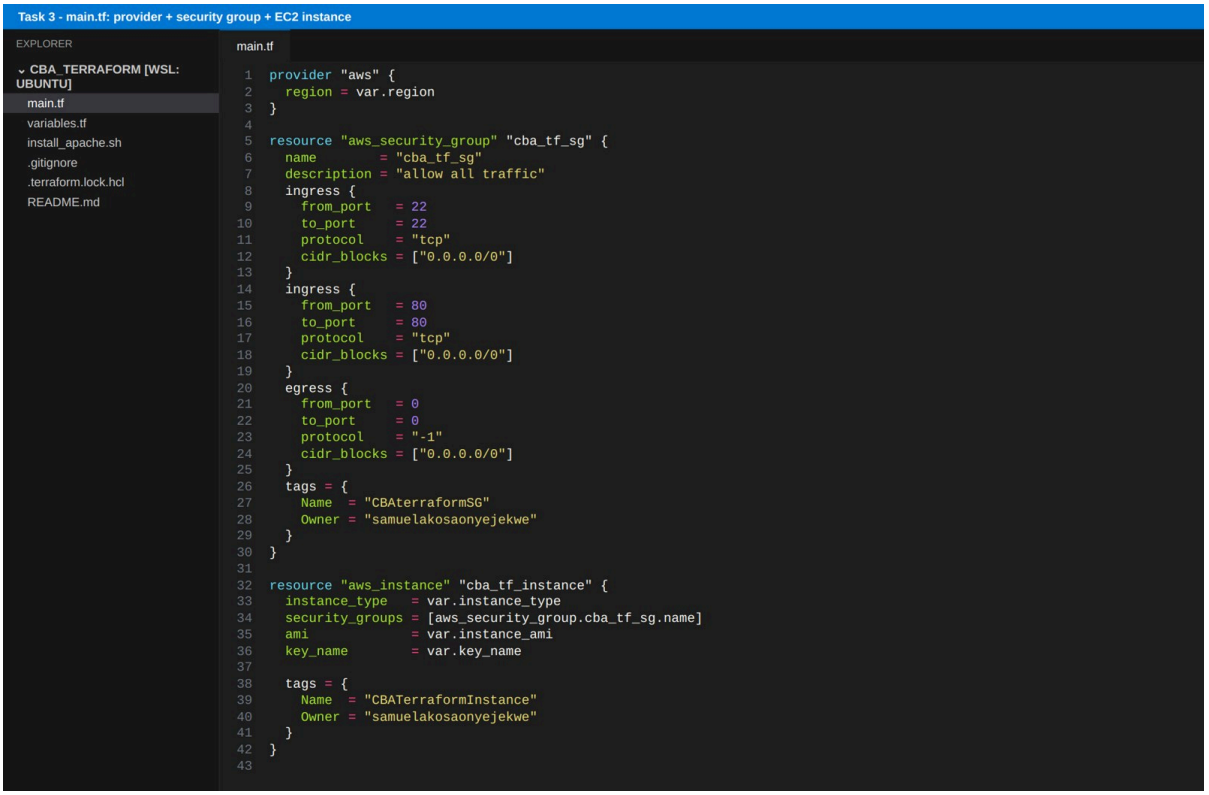


Figure 8: *Initial main.tf: provider, security group and EC2 instance*

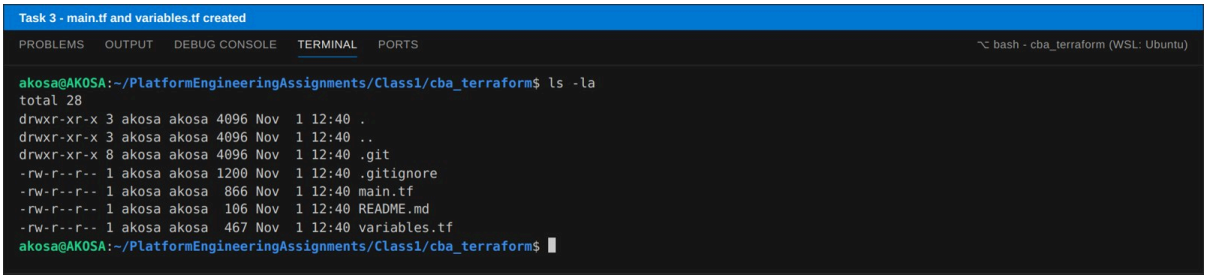


Figure 9: *main.tf and variables.tf created in the cba_terraform folder*

terraform init

I initialised the folder as a Terraform working directory. Terraform downloaded the hashicorp/aws provider (v6.71.0) and created .terraform.lock.hcl to pin the provider version. I also ran terraform fmt and terraform validate to confirm the configuration was well-formed.

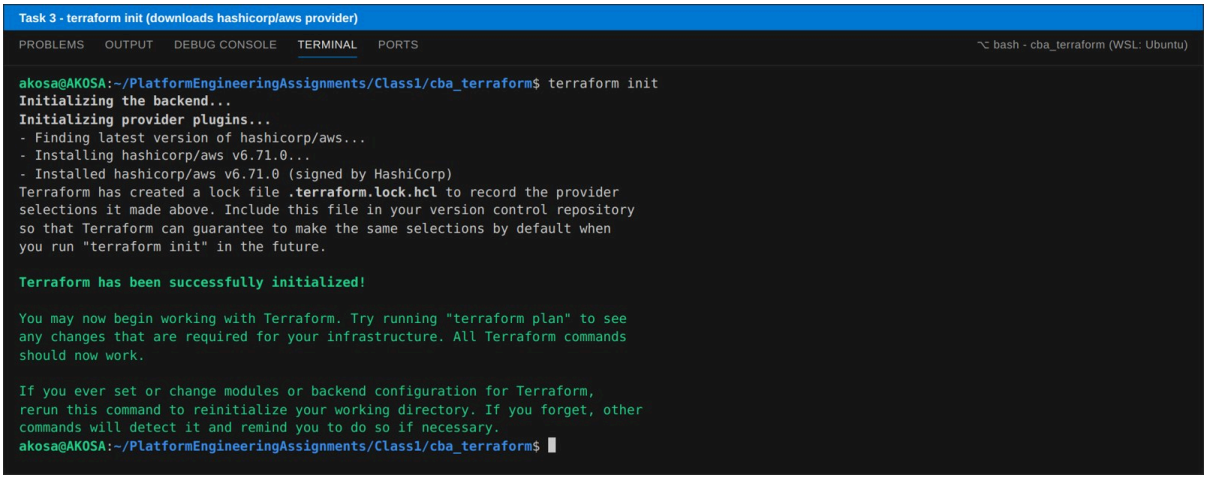


Figure 10: *terraform init – provider installed and directory initialised*

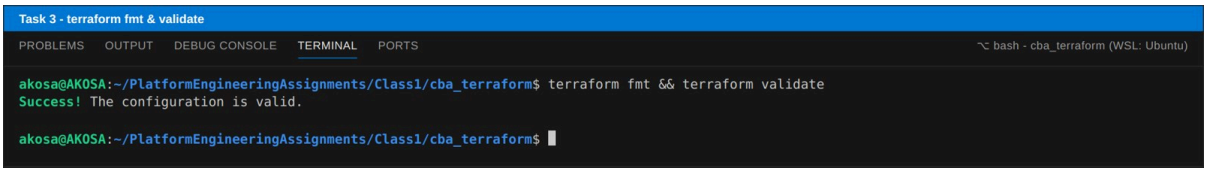


Figure 11: *terraform validate – Success! The configuration is valid*

6. Task 4 – Deploy the Terraform Module to AWS

terraform plan

Before changing anything in AWS I ran terraform plan to preview the execution plan. It showed exactly 2 resources to add (the security group and the instance) and no errors.

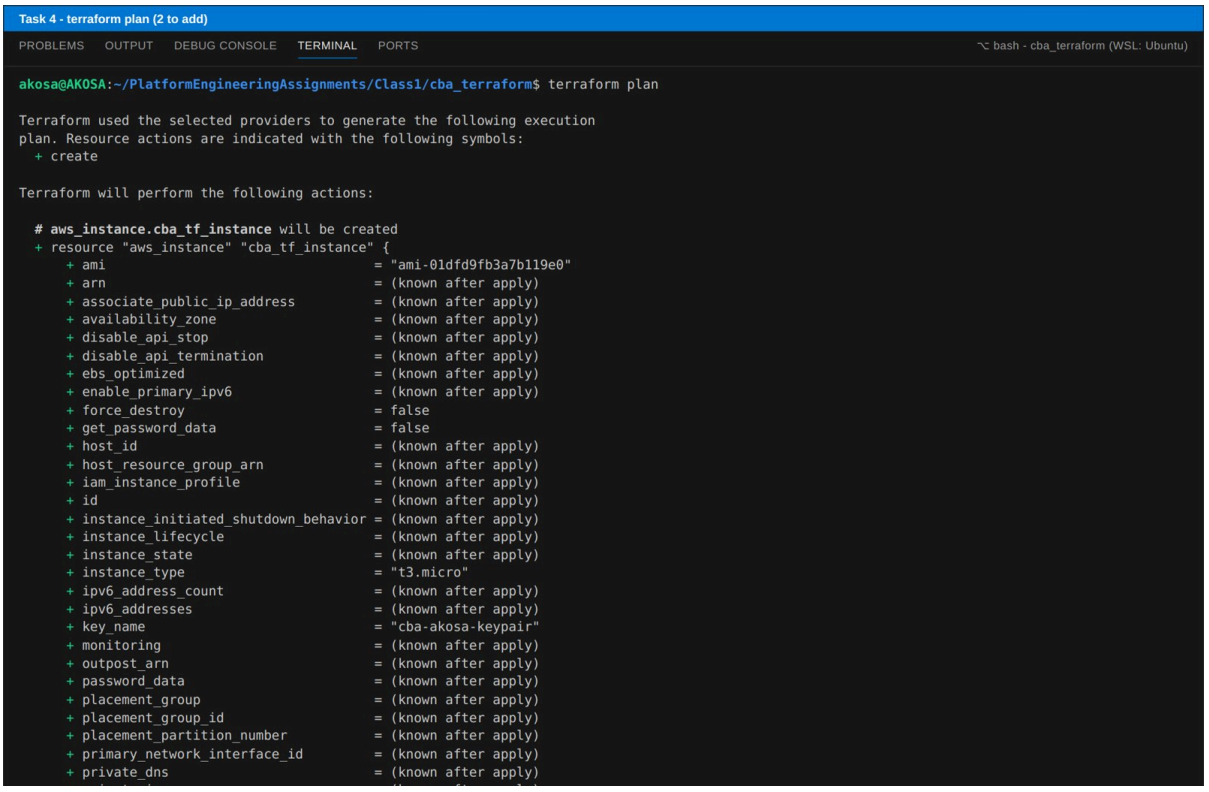


Figure 12: *terraform plan – start of the plan (full output in 01_screenshots/terminal)*

terraform apply

I ran terraform apply, reviewed the plan again and typed yes at the “Enter a value” prompt. Terraform created the security group sg-0a95f125f4f961ac6 in 3 seconds, then the instance i-0aacb5d3808029ccf in 14 seconds: “Apply complete! Resources: 2 added, 0 changed, 0 destroyed.”

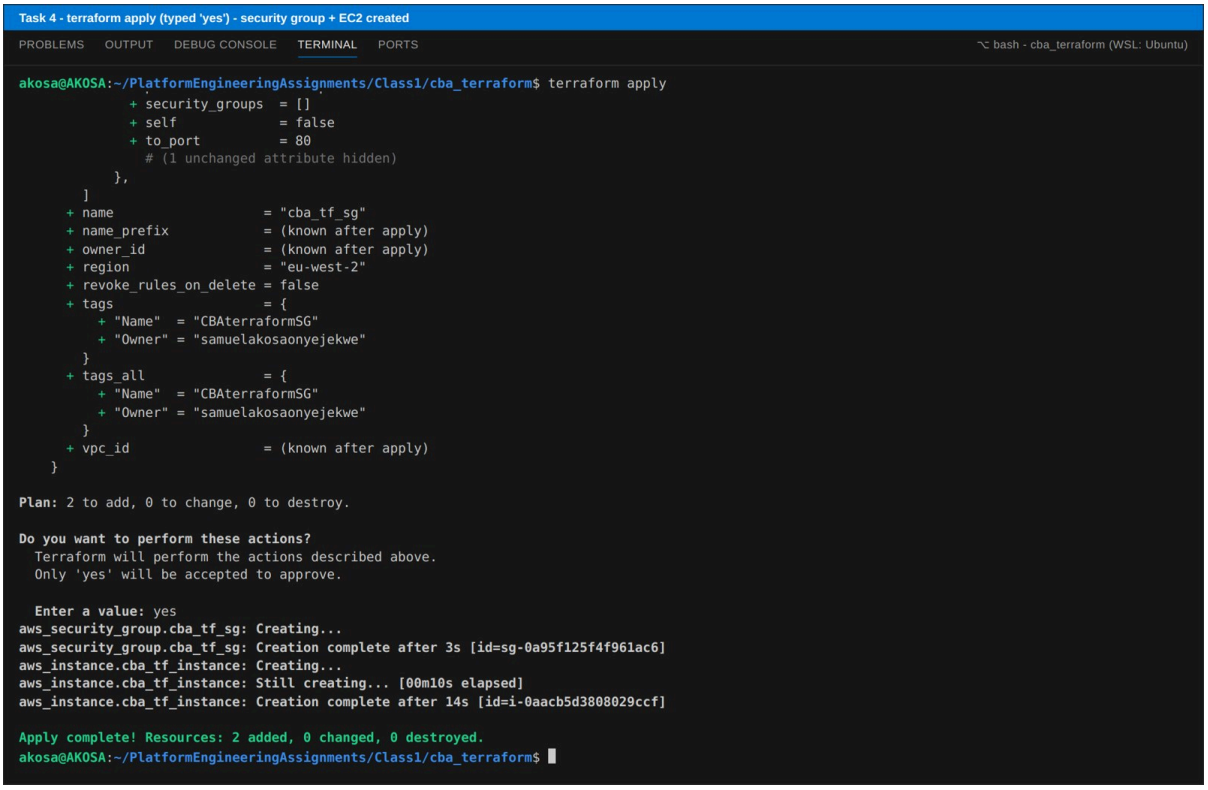


Figure 13: *terraform apply – confirmed with yes; 2 resources added*

Verifying in the AWS Console

I signed in to the AWS Console (London region) and checked the new resources. The EC2 dashboard showed 1 running instance, and the Instances page showed CBATerraformInstance running as a t3.micro and passing its 3/3 status checks.

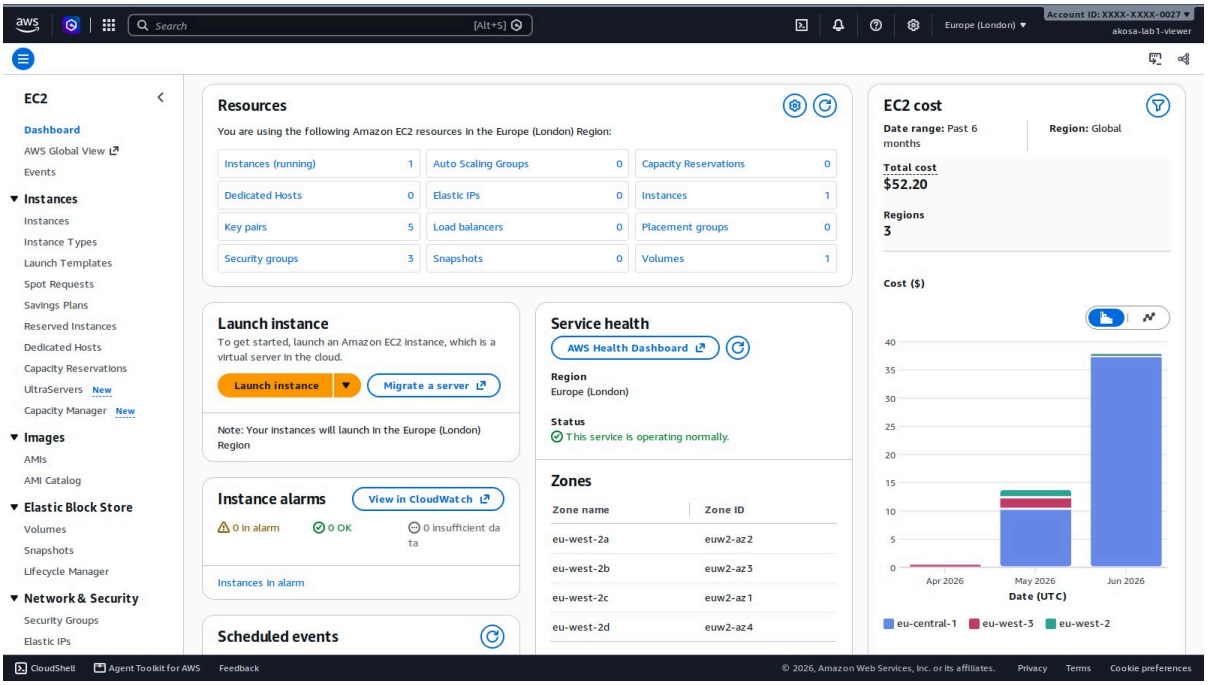


Figure 14: *EC2 dashboard (eu-west-2) – 1 running instance after terraform apply*

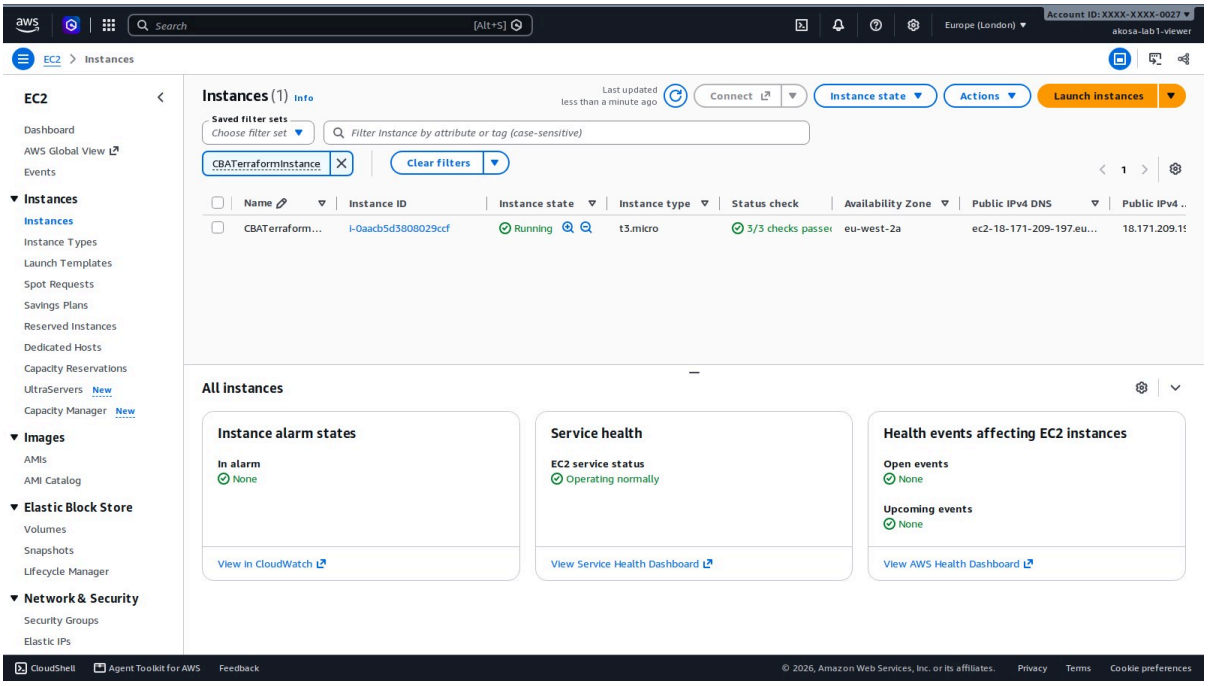


Figure 15: *CBATerraformInstance running, t3.micro, 3/3 checks passed*

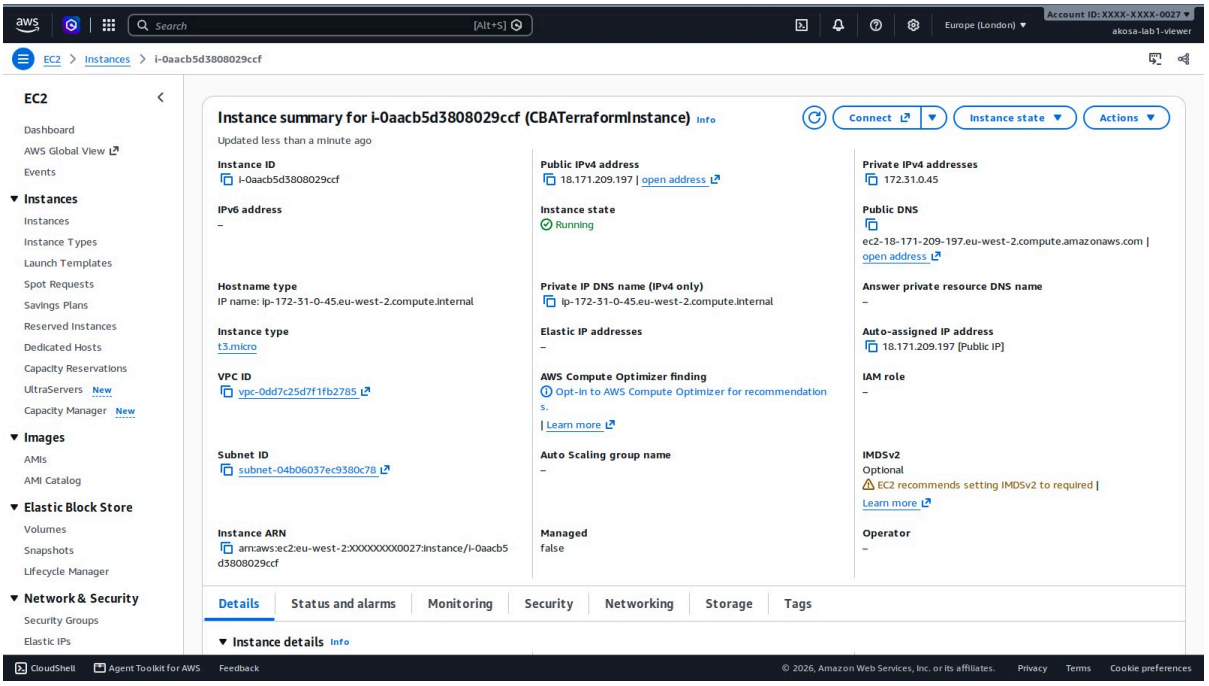


Figure 16: *Instance summary – public IPv4, default VPC and subnet*

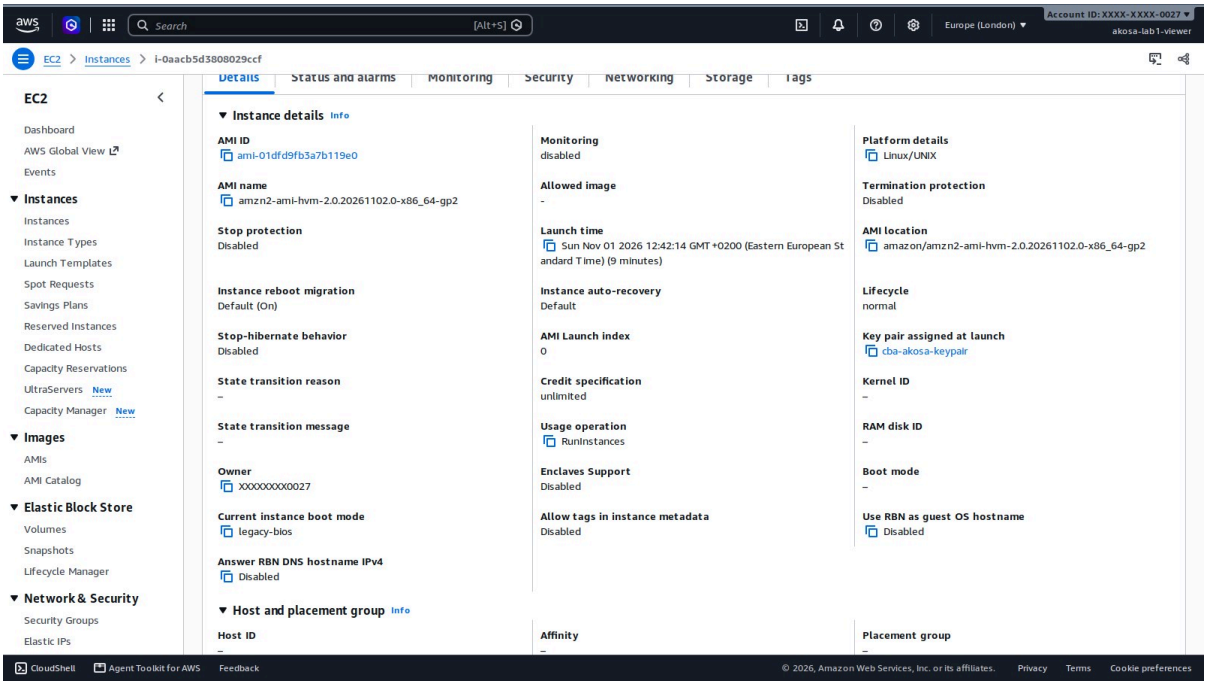


Figure 17: *Instance details – Amazon Linux 2 AMI and my key pair cba-akosa-keypair*

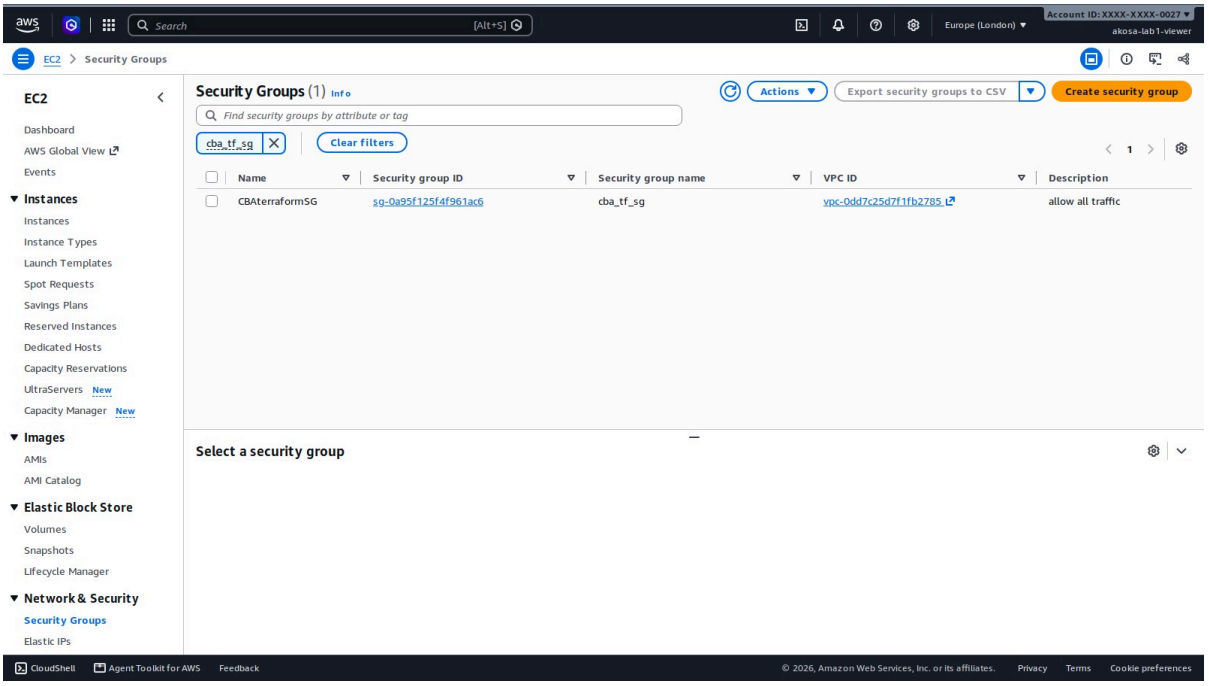


Figure 18: *Security group cba_tf_sg (Name tag CBAterraformSG) – "allow all traffic"*

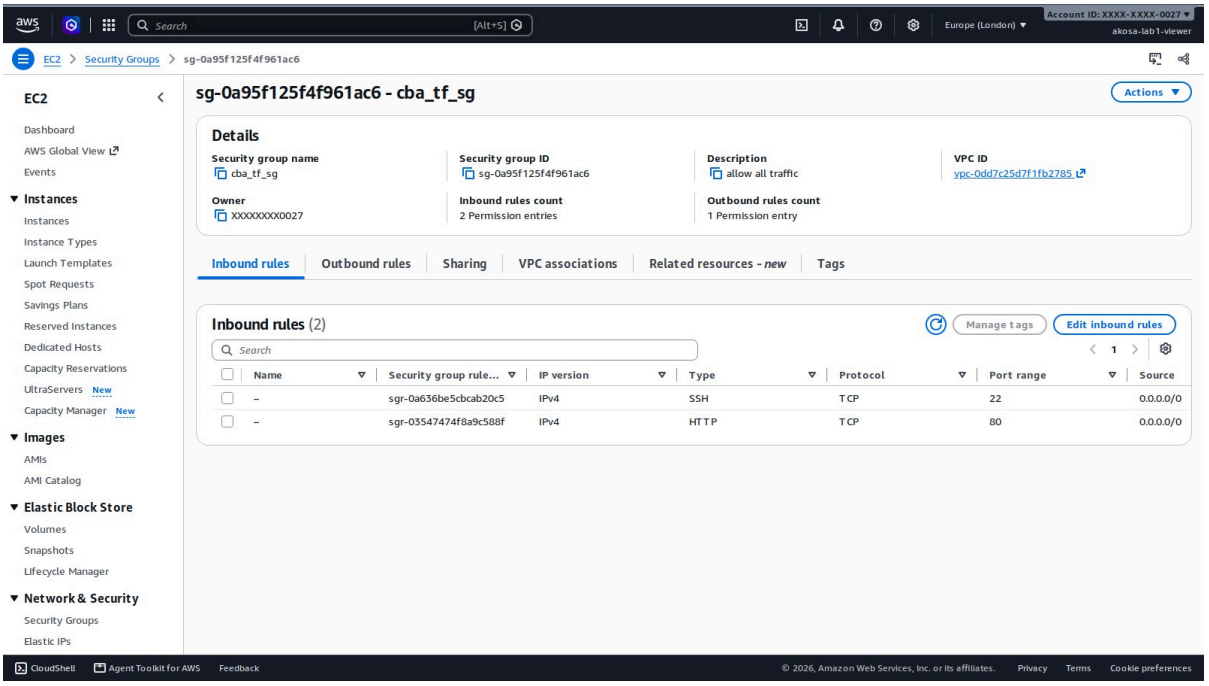


Figure 19: *cba_tf_sg inbound rules – SSH 22 and HTTP 80 from 0.0.0.0/0*

Inspecting the Terraform state

Back in VS Code I inspected the new state. Terraform had written terraform.tfstate, which maps my two resource addresses to the real AWS IDs. terraform state show displayed every attribute of the instance. I also cross-checked the instance with the AWS CLI.

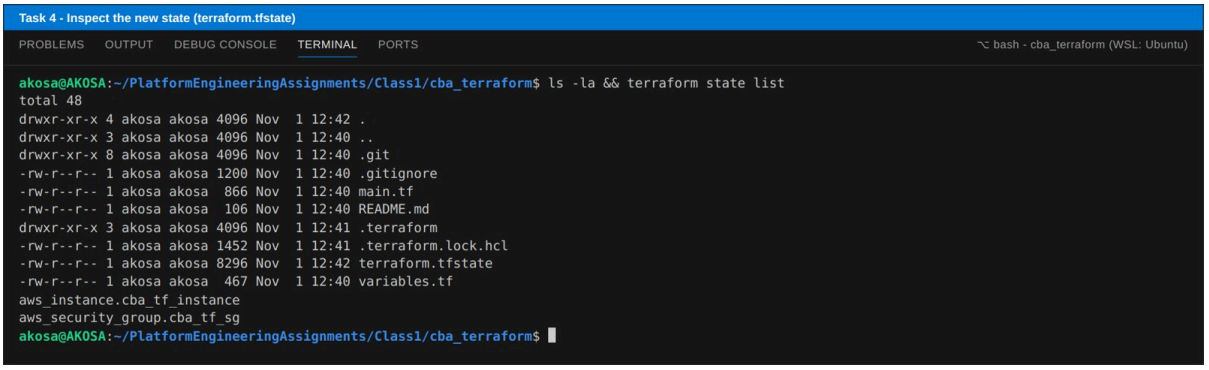


Figure 20: *terraform.tfstate created; terraform state list shows both resources*

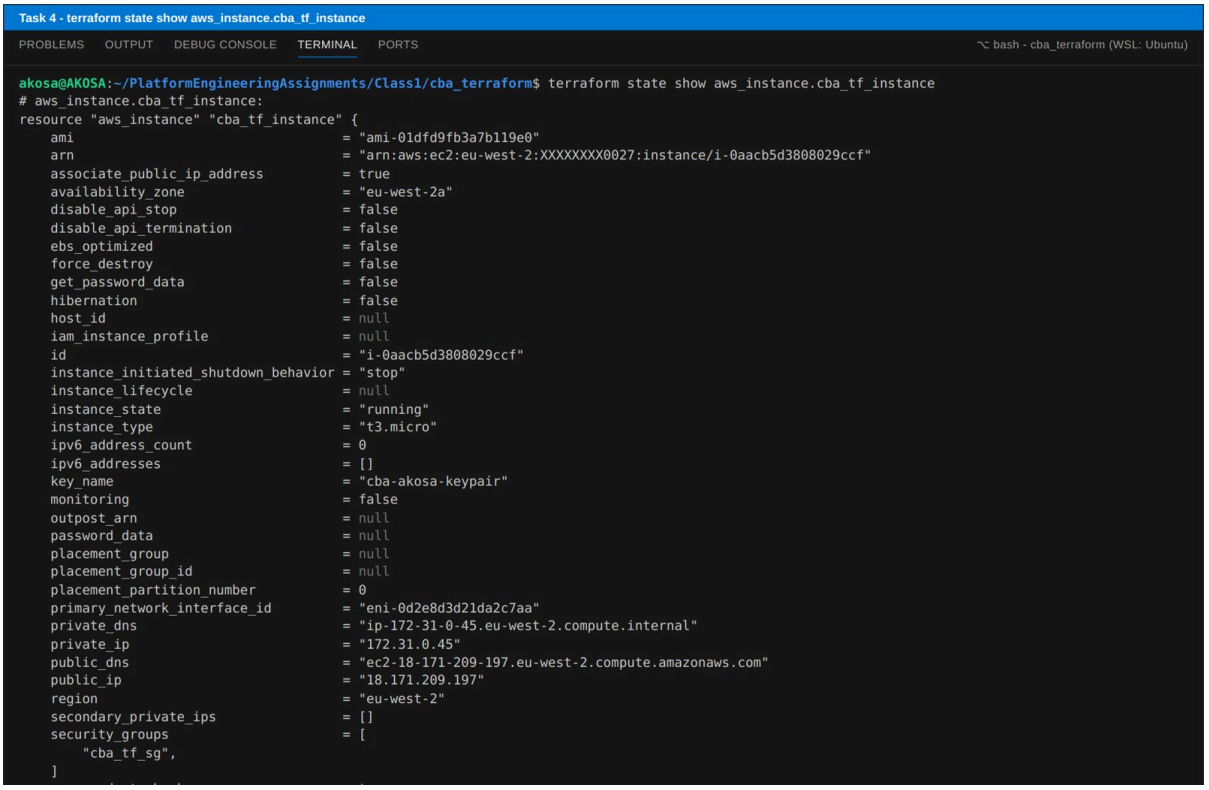


Figure 21: *terraform state show aws_instance.cba_tf_instance (top of output)*

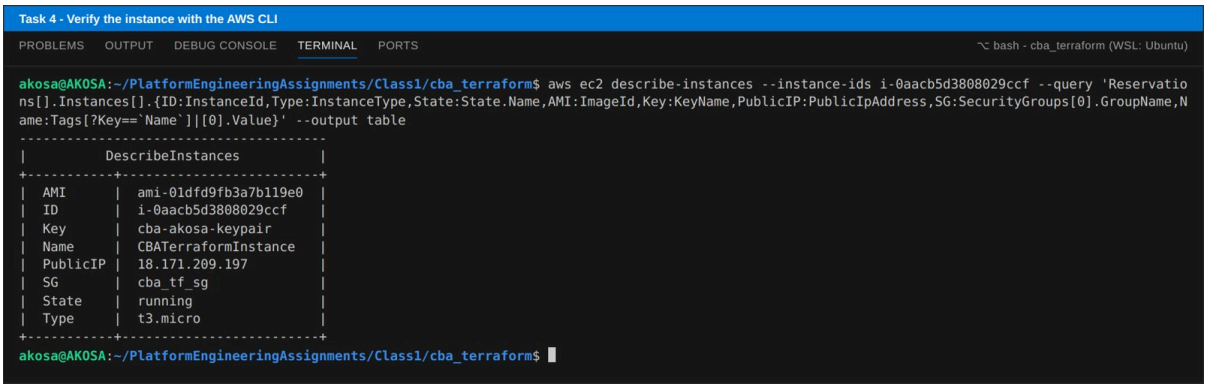


Figure 22: *AWS CLI confirms the instance: t3.micro, running, cba_tf_sg, cba-akosa-keypair*

7. Task 5 – Deploy the Apache Web Server

Next I turned the instance into a web server. I created install_apache.sh, which updates the OS, installs httpd (Apache), starts the service and writes a Hello World page that includes the server's hostname.

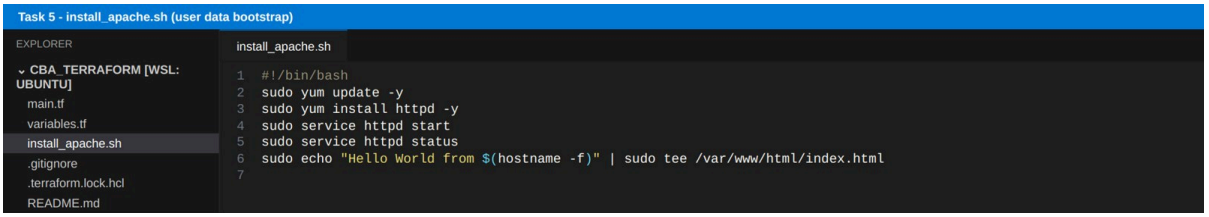


Figure 23: *install_apache.sh – the bootstrap script*

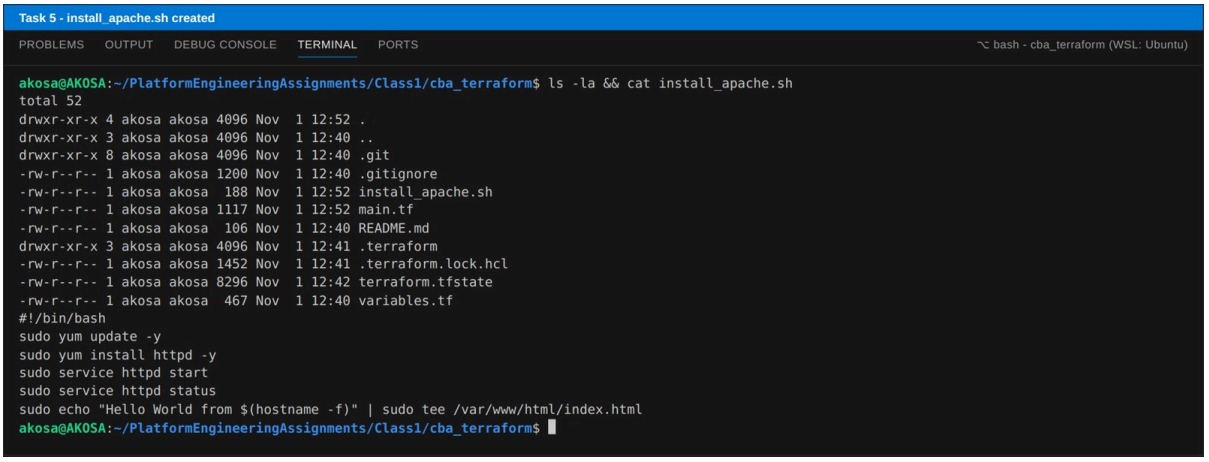


Figure 24: *install_apache.sh created alongside the Terraform files*

I then updated the instance resource in main.tf with user_data = file("install_apache.sh"), so the script is passed to cloud-init when the instance boots.

A problem I solved: user data only runs on first boot

The lab's user_data change is enough on older AWS providers. On provider v5 and later, changing user_data updates the existing instance in place, but cloud-init only runs user data on the first boot, so Apache would never be installed. To fix this I added user_data_replace_on_change = true, which makes Terraform destroy and recreate the instance whenever the script changes. terraform plan confirmed this: "user_data … # forces replacement" and "Plan: 1 to add, 0 to change, 1 to destroy".

Task 5 - main.tf updated with user_data

EXPLORER

CBA_TERRAFORM [WSL: UBUNTU]

- main.tf
- variables.tf
- install_apache.sh
- .gitignore
- .terraform.lock.hcl
- README.md

main.tf

```
provider "aws" {
  region = var.region
}

resource "aws_security_group" "cba_tf_sg" {
  name        = "cba_tf_sg"
  description = "allow all traffic"
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = {
    Name  = "CBAterraformSG"
    Owner = "samuelakosaonyejekwe"
  }
}

resource "aws_instance" "cba_tf_instance" {
  instance_type   = var.instance_type
  security_groups = [aws_security_group.cba_tf_sg.name]
  ami             = var.instance_ami
  key_name        = var.key_name
  user_data       = file("install_apache.sh")

  # user_data only runs on first boot, so force a replacement when it changes
  # (AWS provider v5+ would otherwise update it in place and Apache would never install)
  user_data_replace_on_change = true

  tags = {
    Name  = "CBATerraformInstance"
    Owner = "samuelakosaonyejekwe"
  }
}

```

Figure 25: *main.tf updated with user_data and user_data_replace_on_change*

Task 5 - terraform plan after adding user_data (instance replaced)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ⌥ bash - cba_terraform (WSL: Ubuntu)

```
akosa@AKOSA:~/PlatformEngineeringAssignments/Class1/cba_terraform$ terraform plan
aws_security_group.cba_tf_sg: Refreshing state... [id=sg-0a95f125f4f961ac6]
aws_instance.cba_tf_instance: Refreshing state... [id=i-0aacb5d3808029ccf]

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # aws_instance.cba_tf_instance must be replaced
-/+ resource "aws_instance" "cba_tf_instance" {
      ~ arn                                  = "arn:aws:ec2:eu-west-2:XXXXXXXX0027:instance/i-0aacb5d3808029ccf" -> (known after apply)
      ~ associate_public_ip_address          = true -> (known after apply)
      ~ availability_zone                    = "eu-west-2a" -> (known after apply)
      ~ disable_api_stop                     = false -> (known after apply)
      ~ disable_api_termination              = false -> (known after apply)
      ~ ebs_optimized                        = false -> (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      - hibernation                          = false -> null
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      ~ id                                   = "i-0aacb5d3808029ccf" -> (known after apply)
      ~ instance_initiated_shutdown_behavior = "stop" -> (known after apply)
      + instance_lifecycle                   = (known after apply)
      ~ instance_state                       = "running" -> (known after apply)
      ~ ipv6_address_count                   = 0 -> (known after apply)
      ~ ipv6_addresses                       = [] -> (known after apply)
      ~ monitoring                           = false -> (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      ~ placement_partition_number           = 0 -> (known after apply)
      ~ primary_network_interface_id         = "eni-0d2e8d3d21da2c7aa" -> (known after apply)
      ~ private_dns                          = "ip-172-31-0-45.eu-west-2.compute.internal" -> (known after apply)
      ~ private_ip                           = "172.31.0.45" -> (known after apply)
      ~ public_dns                           = "ec2-18-171-209-197.eu-west-2.compute.amazonaws.com" -> (known after apply)
```

Figure 26: *terraform plan – the instance must be replaced because user_data changed*

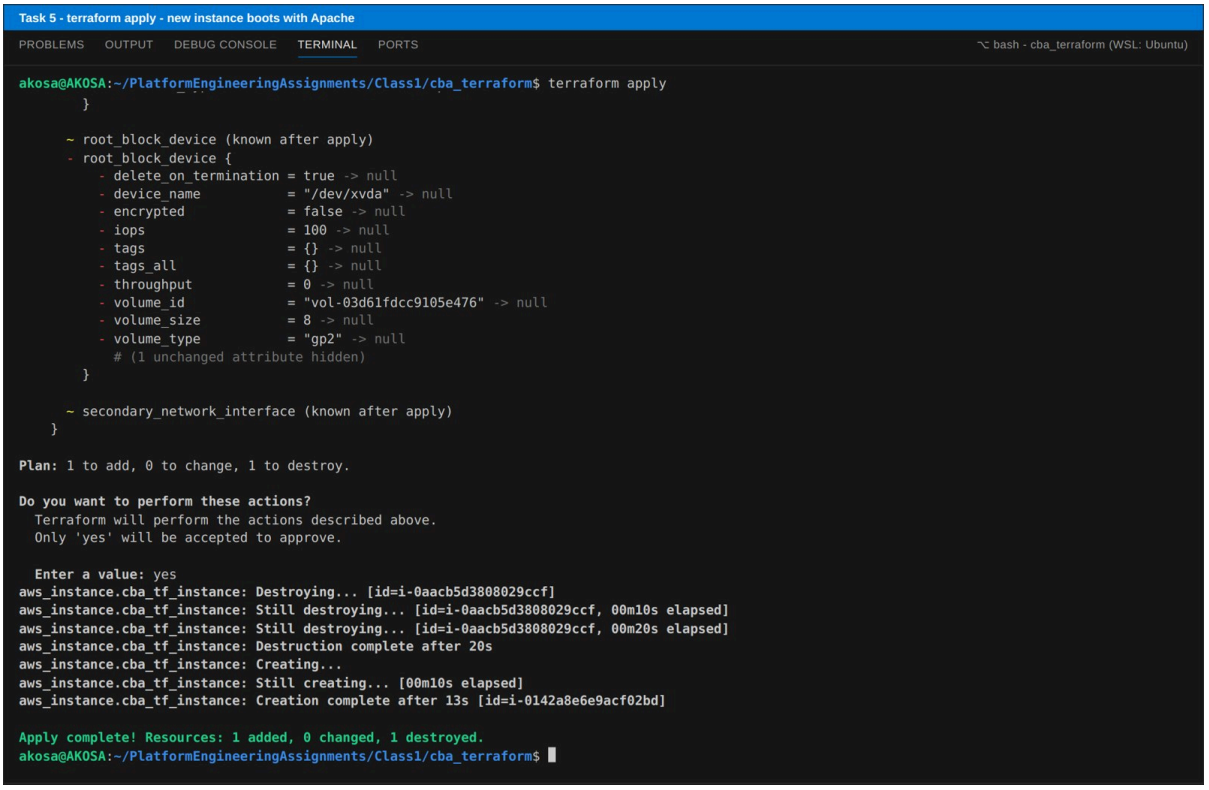


Figure 27: *terraform apply – old instance destroyed, new instance i-0142a8e6e9acf02bd created*

Testing the web server

After about a minute, once cloud-init had finished, I took the new instance's public IP (52.56.89.180) from the Terraform state and the console. I opened it in my browser and also tested it with curl. Apache answered with HTTP/1.1 200 OK and the page "Hello World from ip-172-31-15-40.eu-west-2.compute.internal".

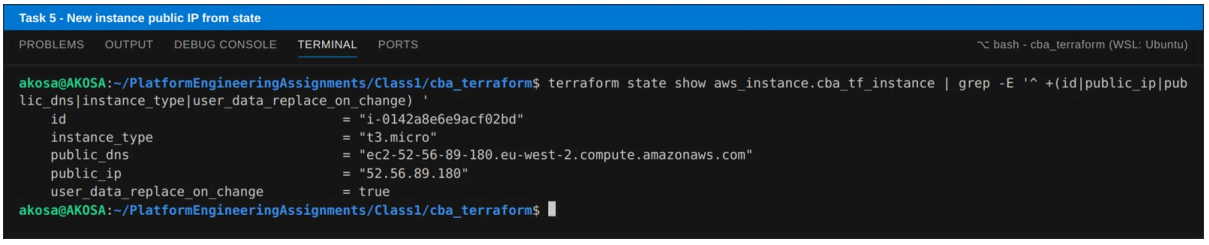


Figure 28: *New instance public IP and DNS from terraform state*

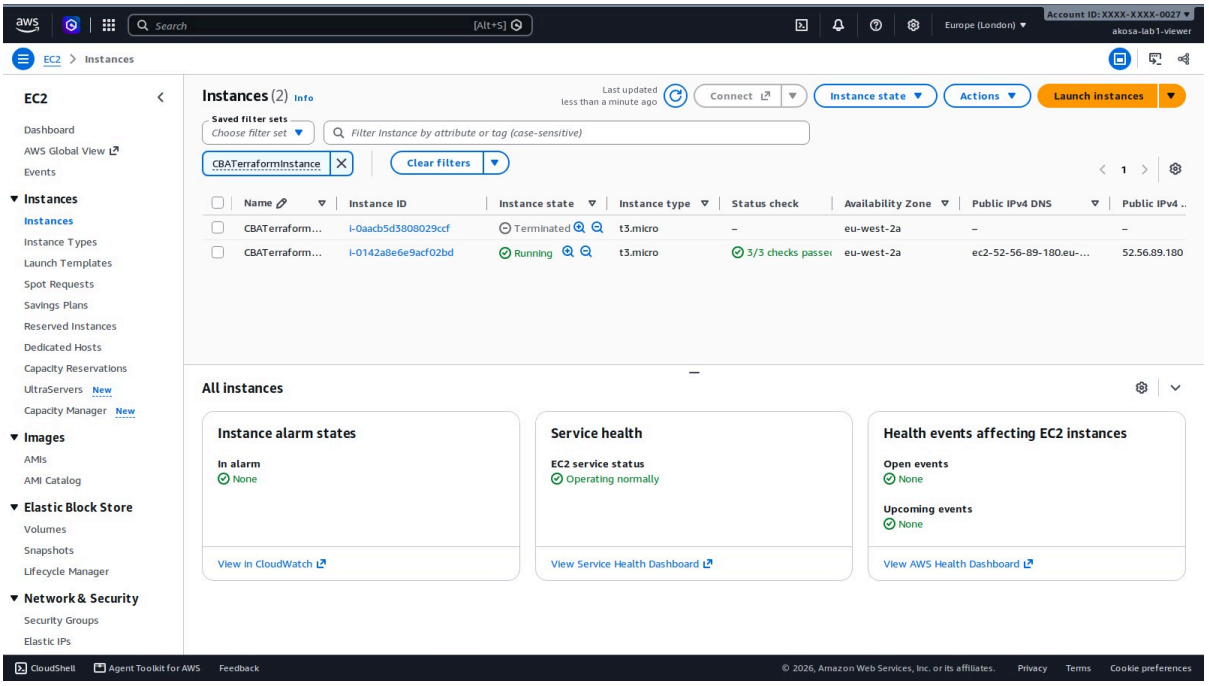


Figure 29: *Console – the old instance terminated and the new Apache instance running*

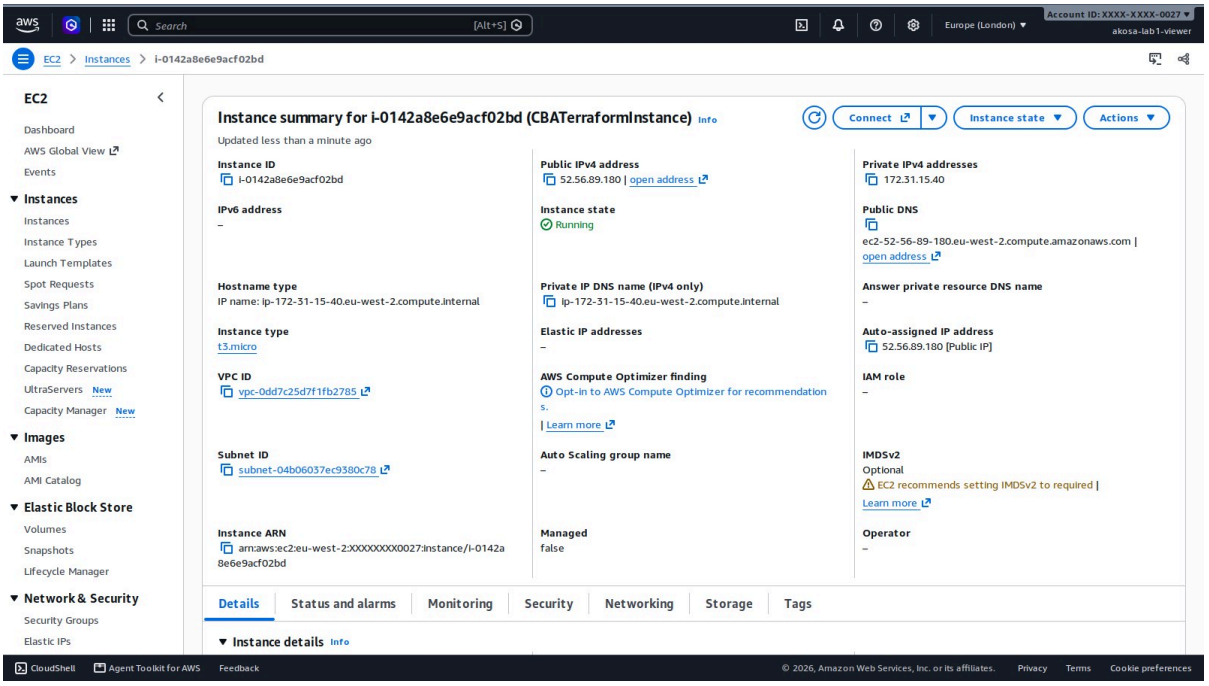


Figure 30: *Console – new instance summary with public IPv4 52.56.89.180*



Figure 31: *Browser – http://52.56.89.180 shows Hello World from my EC2 instance*

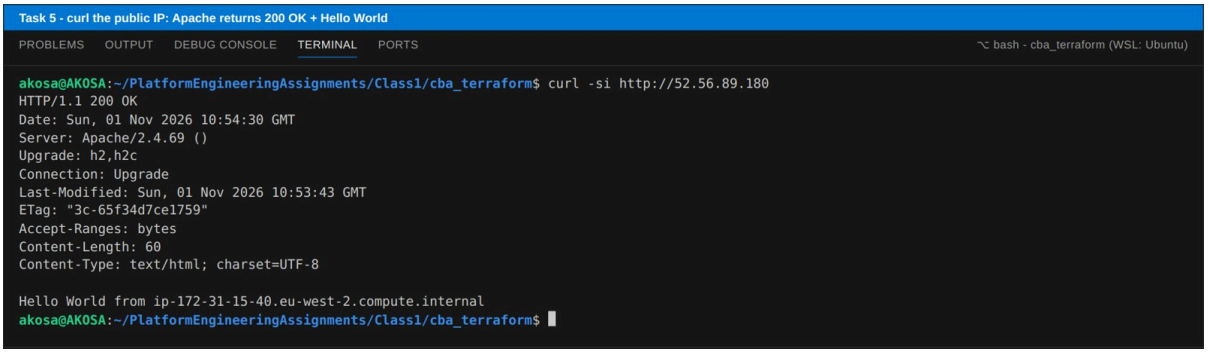


Figure 32: *curl – HTTP/1.1 200 OK from Apache/2.4.69*

8. Task 6 – Push the Terraform Files to GitHub

With the module working, I pushed my code to the cba_terraform repository. Thanks to the .gitignore, terraform.tfstate and the .terraform/ directory were automatically excluded. Only the configuration files, the provider lock file and the README were committed. I expanded the README to document what the module provisions and how to use it.

5. git status – showed the untracked .tf files, install_apache.sh, .terraform.lock.hcl and the modified README.
6. git add . then git status – all files staged.
7. git commit -m "Lab 1: Terraform module to provision EC2 + security group and deploy Apache web server"
8. git push – pushed commit 3a98d6f to origin/main.

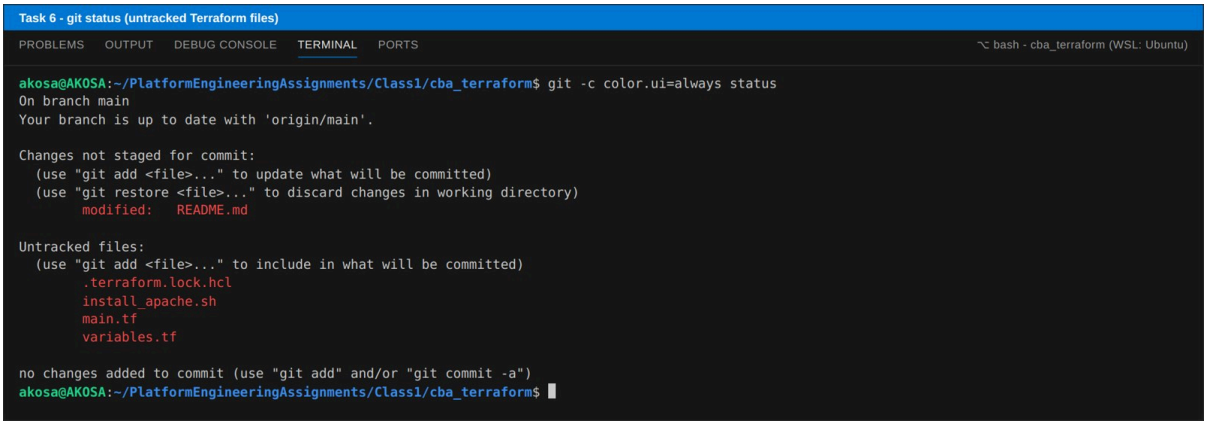


Figure 33: *git status – untracked files (terraform.tfstate is ignored)*

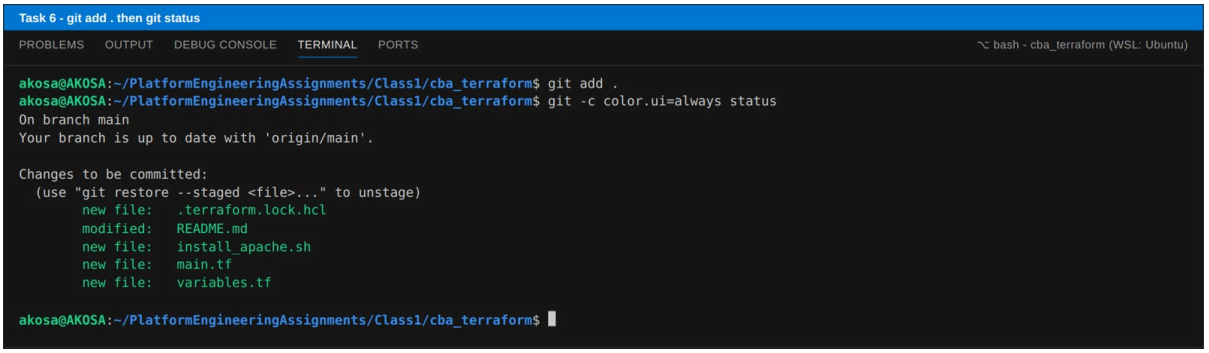


Figure 34: *git add . followed by git status*

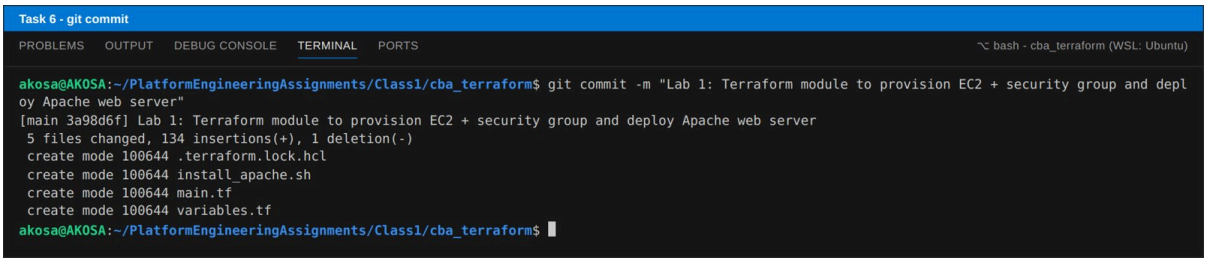


Figure 35: *git commit – 5 files changed*

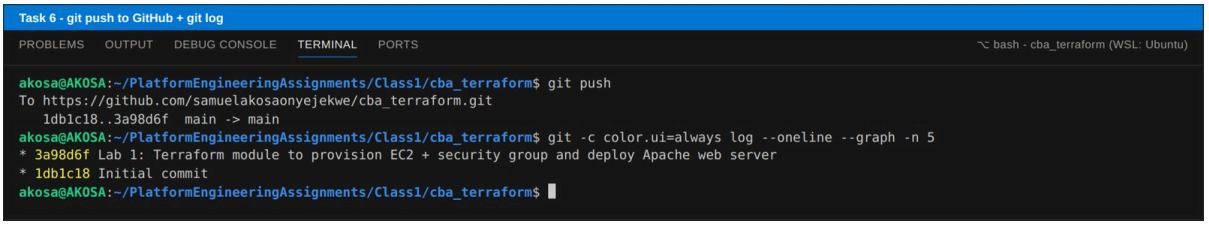


Figure 36: *git push to GitHub and the commit history*

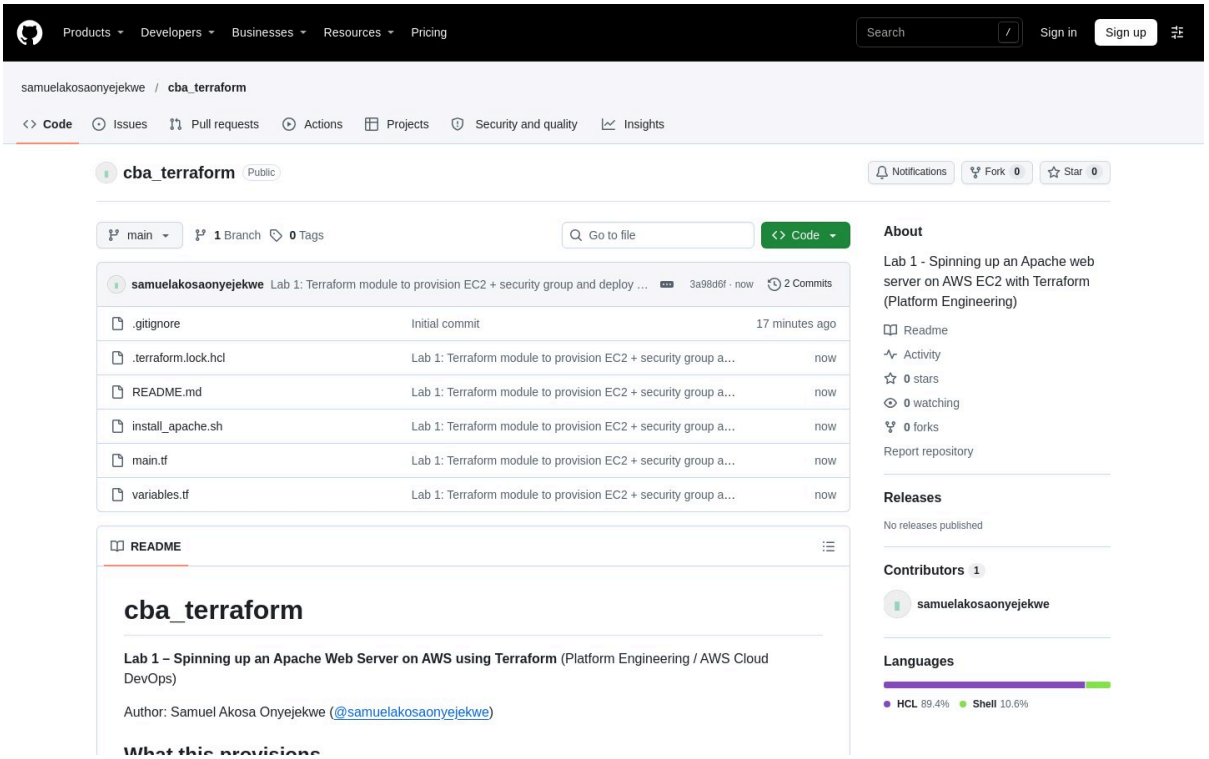
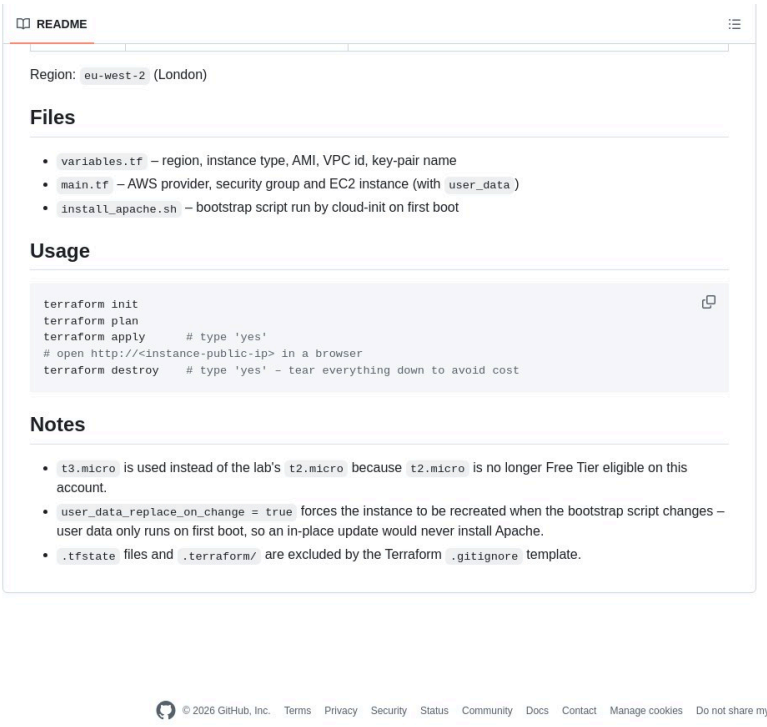


Figure 37: *GitHub – cba_terraform repository with my pushed files*



README

Region: `eu-west-2` (London)

Files

- `variables.tf` – region, instance type, AMI, VPC id, key-pair name
- `main.tf` – AWS provider, security group and EC2 instance (with `user_data`)
- `install_apache.sh` – bootstrap script run by cloud-init on first boot

Usage

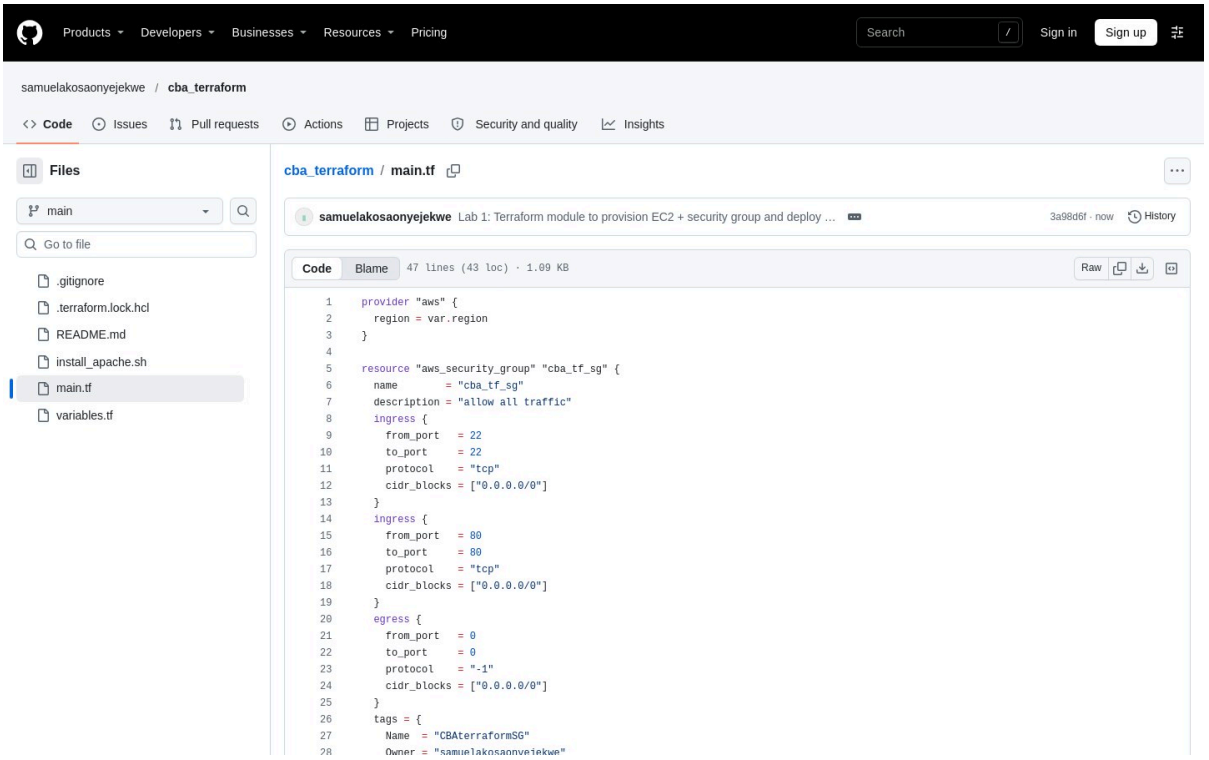
```
terraform init
terraform plan
terraform apply      # type 'yes'
# open http://<instance-public-ip> in a browser
terraform destroy    # type 'yes' – tear everything down to avoid cost
```

Notes

- `t3.micro` is used instead of the lab's `t2.micro` because `t2.micro` is no longer Free Tier eligible on this account.
- `user_data_replace_on_change = true` forces the instance to be recreated when the bootstrap script changes – user data only runs on first boot, so an in-place update would never install Apache.
- `.tfstate` files and `.terraform/` are excluded by the Terraform `.gitignore` template.

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Figure 38: *GitHub – README documenting the module*



Products Developers Businesses Resources Pricing Search / Sign in Sign up

samuelakosaonyejekwe / cba_terraform

Code Issues Pull requests Actions Projects Security and quality Insights

Files

main

Go to file

.gitignore
.terraform.lock.hcl
README.md
install_apache.sh
main.tf
variables.tf

cba_terraform / main.tf

samuelakosaonyejekwe Lab 1: Terraform module to provision EC2 + security group and deploy … 3a98d6f · now History

Code Blame 47 lines (43 loc) · 1.09 KB Raw

```
1   provider "aws" {
2     region = var.region
3   }
4
5   resource "aws_security_group" "cba_tf_sg" {
6     name        = "cba_tf_sg"
7     description = "allow all traffic"
8     ingress {
9       from_port   = 22
10      to_port     = 22
11      protocol    = "tcp"
12      cidr_blocks = ["0.0.0.0/0"]
13    }
14    ingress {
15      from_port   = 80
16      to_port     = 80
17      protocol    = "tcp"
18      cidr_blocks = ["0.0.0.0/0"]
19    }
20    egress {
21      from_port   = 0
22      to_port     = 0
23      protocol    = "-1"
24      cidr_blocks = ["0.0.0.0/0"]
25    }
26    tags = {
27      Name  = "CBAterraformSG"
28      Owner = "samuelakosaonyejekwe"
```

Figure 39: *GitHub – main.tf*

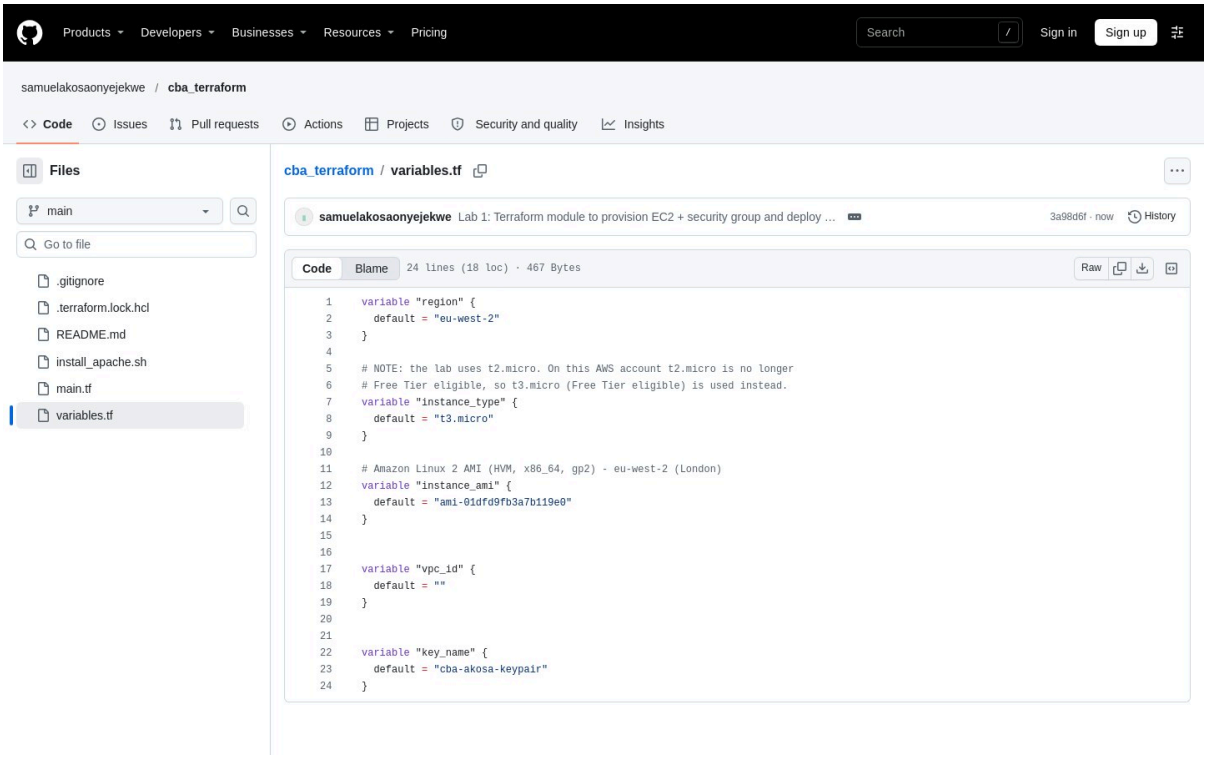


Figure 40: *GitHub – variables.tf*

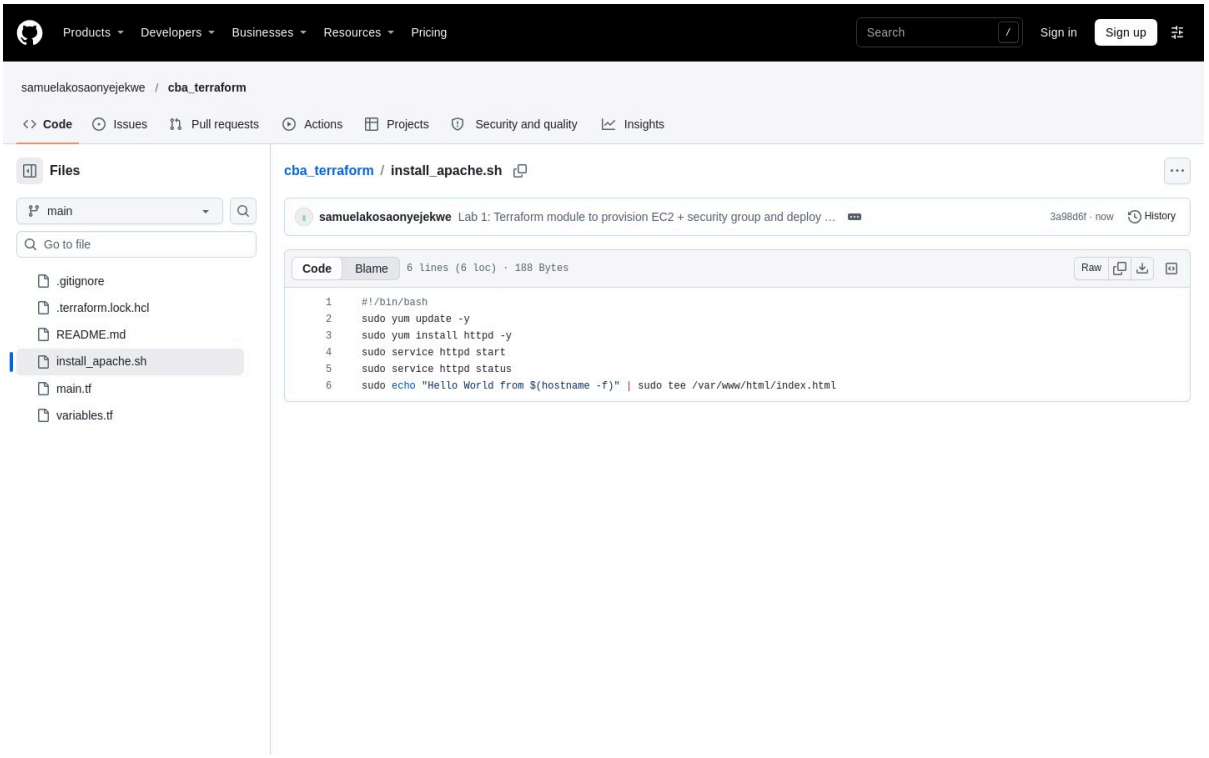


Figure 41: *GitHub – install_apache.sh*

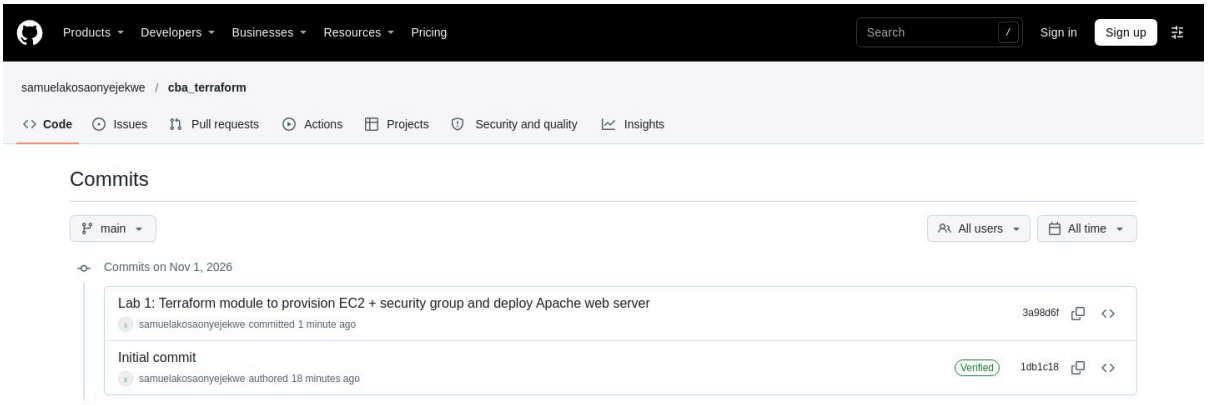


Figure 42: *GitHub – commit history*

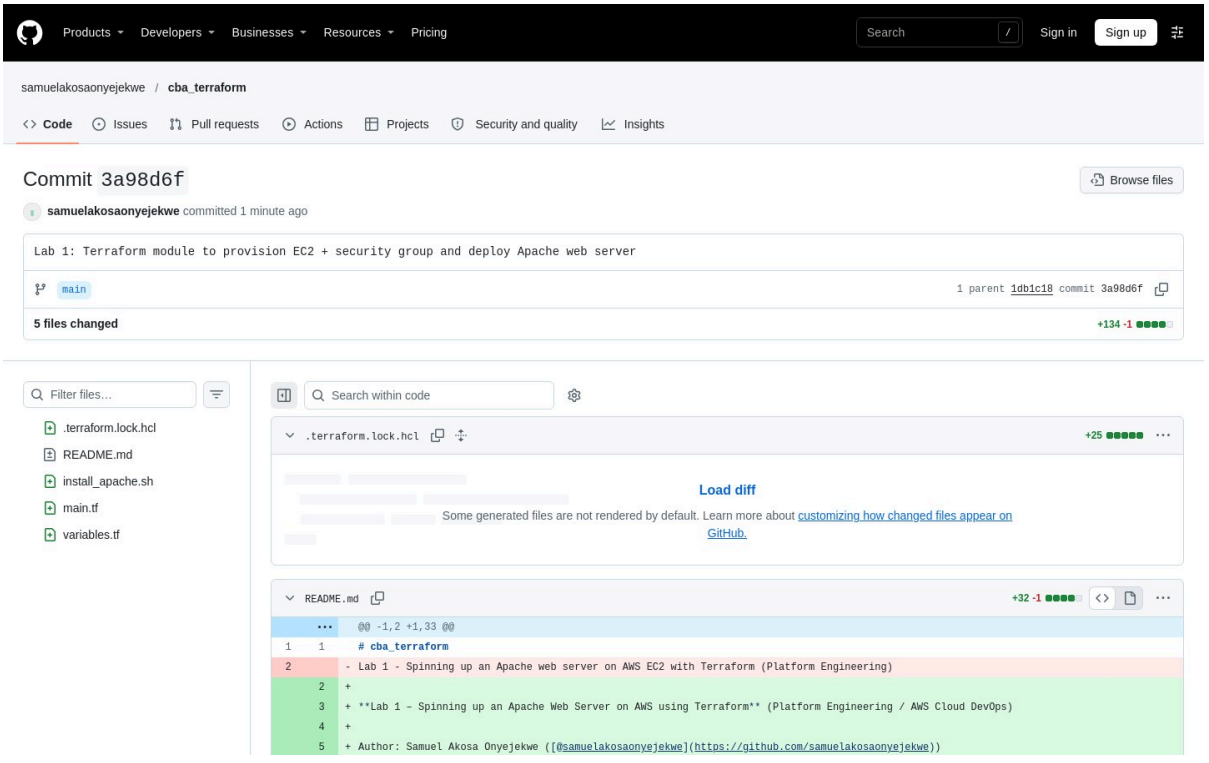


Figure 43: *GitHub – the Lab 1 commit diff*

9. Task 7 – Destroy the Resources

To avoid any ongoing cost I ran terraform destroy and typed yes. Terraform respected the dependency order: it terminated the instance first (21 s) and then deleted the security group: “Destroy complete! Resources: 2 destroyed.”

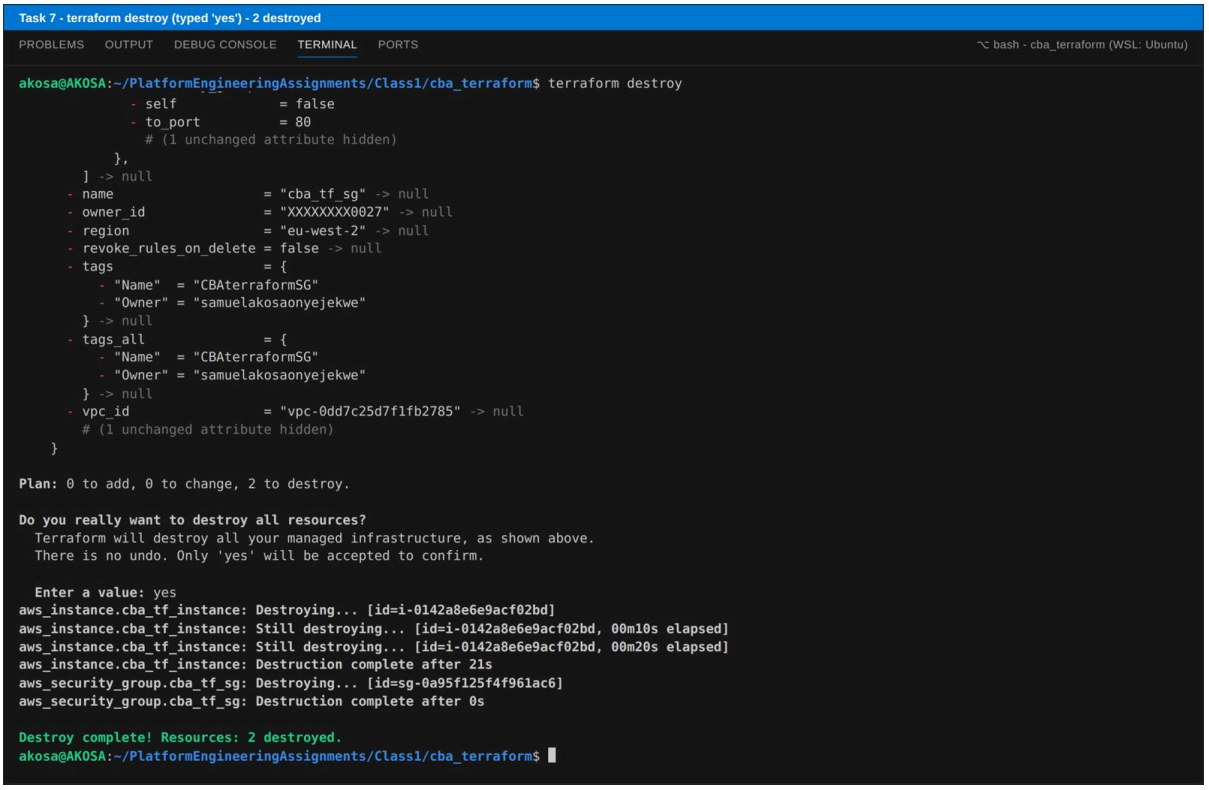


Figure 44: *terraform destroy – 2 resources destroyed*

I then verified the cleanup three ways. The Terraform state is now empty. The AWS CLI shows both lab instances terminated, no cba_tf_sg security group, no running or stopped instances, no unattached EBS volumes and no Elastic IPs in eu-west-2. The AWS Console agrees.

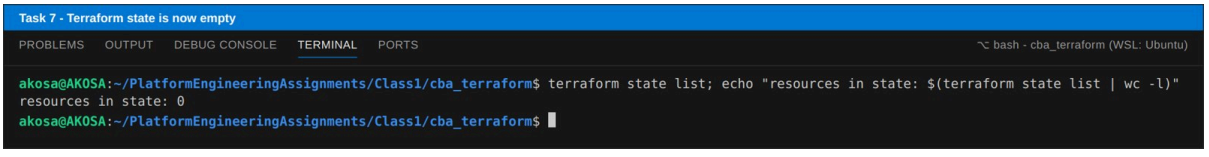


Figure 45: *Terraform state is empty after destroy*

```
Task 7 - AWS CLI confirms nothing left running (no cost)
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    ⌥ bash - cba_terraform (WSL: Ubuntu)

akosa@AKOSA:~/PlatformEngineeringAssignments/Class1/cba_terraform$ aws ec2 describe-instances --filters Name=tag:Name,Values=CBATerraformInstance --
query 'Reservations[].Instances[].[InstanceId,State.Name]' --output table; echo '--- security groups named cba_tf_sg:'; aws ec2 describe-security-gr
oups --filters Name=group-name,Values=cba_tf_sg --query 'length(SecurityGroups)'; echo '--- running instances in eu-west-2:'; aws ec2 describe-insta
nces --filters Name=instance-state-name,Values=pending,running,stopping,stopped --query 'length(Reservations[].Instances[])'; echo '--- unattached E
BS volumes:'; aws ec2 describe-volumes --filters Name=status,Values=available --query 'length(Volumes)'; echo '--- Elastic IPs:'; aws ec2 describe-a
ddresses --query 'length(Addresses)'
--------------------------------------
|          DescribeInstances          |
+---------------------+--------------+
|  i-0aacb5d3808029ccf |  terminated  |
|  i-0142a8e6e9acf02bd |  terminated  |
+---------------------+--------------+
--- security groups named cba_tf_sg:
0
--- running instances in eu-west-2:
0
--- unattached EBS volumes:
0
--- Elastic IPs:
0
akosa@AKOSA:~/PlatformEngineeringAssignments/Class1/cba_terraform$
```

Figure 46: *AWS CLI – nothing left running, so no further cost*

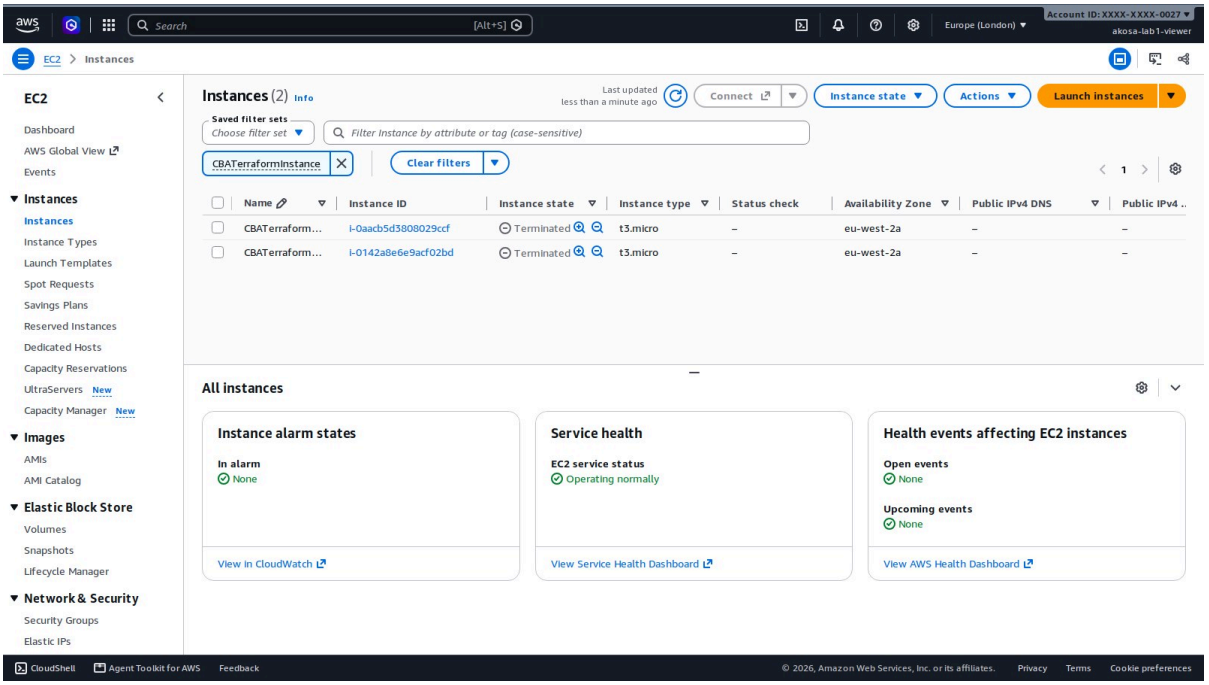


Figure 47: *Console – both CBATerraformInstance instances terminated*

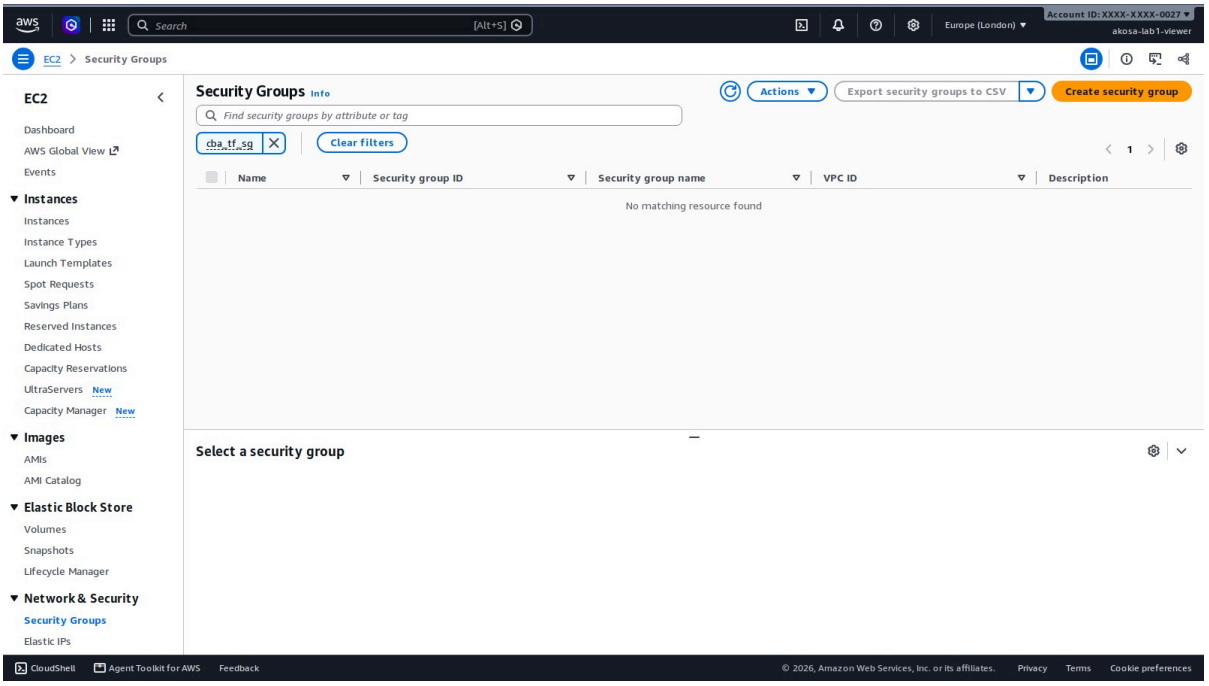


Figure 48: *Console – cba_tf_sg no longer exists*

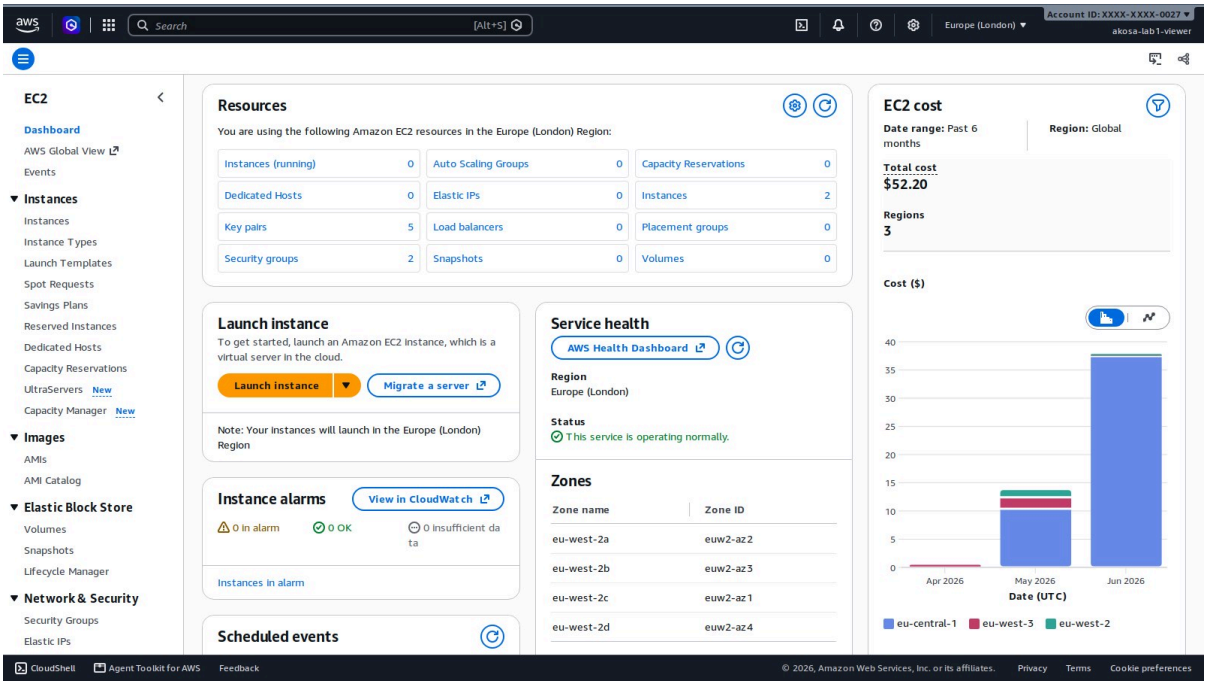


Figure 49: *Console – EC2 dashboard shows 0 running instances*

10. Challenges, Decisions and Key Learnings

Challenges and how I resolved them

Challenge	Resolution
t2.micro (used in the lab) is no longer Free Tier eligible on my account	I queried the Free Tier eligible instance types with the AWS CLI and switched to t3.micro, which is eligible, so the lab cost nothing.
Adding user_data to a running instance does not re-run the script (AWS provider v6 updates user_data in place)	I added user_data_replace_on_change = true so Terraform recreates the instance and cloud-init runs install_apache.sh on first boot.
Choosing the right AMI	I resolved the latest Amazon Linux 2 AMI for eu-west-2 from the public SSM parameter instead of the default Amazon Linux 2023 AMI, as the lab instructs.
Keeping secrets and state out of Git	I used the Terraform .gitignore template, so terraform.tfstate, .terraform/ and *.tfvars are never committed.

What I learned

- Infrastructure as Code: the whole environment (firewall, server and web application bootstrap) is described in a few small, reviewable, version-controlled files.
- The Terraform workflow: init → (fmt/validate) → plan → apply → inspect state → destroy. terraform plan is a safety net that shows exactly what will change before anything happens.
- Resource references (aws_security_group.cba_tf_sg.name) create implicit dependencies, so Terraform creates and destroys resources in the right order.
- The state file is how Terraform tracks the real infrastructure. It is sensitive and must not be committed to a public repository.
- User data and cloud-init can bootstrap servers automatically at launch, but only on first boot. Understanding provider behaviour (replace vs update in place) matters.
- Cost hygiene: verify Free Tier eligibility before provisioning, and confirm a full teardown afterwards.

Possible improvements

- Restrict SSH (port 22) to my own IP address instead of 0.0.0.0/0, and enforce IMDSv2 on the instance.
- Add outputs.tf (public IP and URL), use vpc_security_group_ids, and store state in a remote S3 backend with locking.

- Add a GitHub Actions pipeline that runs terraform fmt, validate and plan on every pull request.

11. Evidence Index

Everything below is saved in my assignments\lab1 folder.

Folder	Contents
01_screenshots\terminal	24 step-by-step terminal screenshots (full, uncropped command output)
01_screenshots\aws_console	12 AWS Console and browser screenshots (create, Apache, destroy)
01_screenshots\github	9 GitHub screenshots (repo, files, commits, profile)
01_screenshots\code	6 code screenshots (variables.tf, main.tf before/after, install_apache.sh, README, .gitignore)
02_videos	10 screen recordings (.webm) covering every task, in order
03_terraform_code	Final module plus the stage-1 (before user_data) version and the repo link
04_logs	Raw, timestamped output of every command I ran (.txt and .json)

Note: my AWS account ID is masked (XXXX-XXXX-0027) in all screenshots, videos and logs because this report is published in my portfolio.